Reti Logiche

Appunti del corso · a.a. 2026/27 · Mobius

Appunti personali e non ufficiali, rivisti e confrontati con il programma ufficiale del corso. Non sostituiscono le lezioni né i testi consigliati.

Indice

1. Algebra booleana e sintesi delle reti combinatorie
2. Reti combinatorie standard, ritardi e aritmetica
3. Flip-flop, registri e contatori
4. Macchine a stati finiti e progetto con ASM

1 · Algebra booleana e sintesi delle reti combinatorie

In questo modulo. Non servono prerequisiti oltre all'algebra delle superiori. Si introducono le variabili e le funzioni booleane, le porte logiche che le realizzano e l'algebra che permette di trasformare un'espressione in un'altra equivalente. Poi si passa al problema centrale delle reti combinatorie: data una specifica, scritta come tabella di verità, trovare un circuito che la realizzi con il minor numero di porte. Gli strumenti sono le forme canoniche del teorema di Shannon e le mappe di Karnaugh, con le indifferenze e le variabili riportate. Alla fine del modulo si sa passare da una specifica a una sintesi minima a due livelli, nelle due forme AND-OR e OR-AND, e realizzarla anche con sole porte NAND o NOR.

1.1 Variabili e funzioni booleane

Nella matematica classica una funzione $y = f(x)$ associa a un numero reale x un numero reale y. Nelle reti logiche le variabili possono assumere solo due valori, 0 e 1, che si leggono anche come *falso* e *vero*. Due valori sono la quantità minima per portare un'informazione: una variabile di questo tipo si chiama **variabile booleana**, e il suo valore è un **bit**.

> **Funzione booleana**
>
> Sia $\mathbb{B} = \{0, 1\}$. Una **funzione booleana** di n variabili è una funzione $f : \mathbb{B}^n \to \mathbb{B}$, che associa a ogni n-upla $(x_1, \ldots, x_n)$ di bit un bit $y = f(x_1, \ldots, x_n)$. Una funzione con p uscite, $f : \mathbb{B}^n \to \mathbb{B}^p$, è un vettore di p funzioni booleane $y_k = f_k(x_1, \ldots, x_n)$, che si studiano una alla volta.

Poiché ogni variabile ha due valori, le combinazioni degli ingressi sono 2^n (**principio di enumerazione**): una funzione si descrive completamente elencando il valore di uscita per ciascuna di esse, cioè con la sua **tabella di verità**, che ha 2^n righe. Ogni riga può contenere 0 o 1 indipendentemente dalle altre, quindi le funzioni distinte di n variabili sono 2^{2^n}: 4 per $n = 1$, 16 per $n = 2$, 256 per $n = 3$. Sono in numero finito, e per n piccolo anche pochissime.

Una **rete combinatoria** è un circuito digitale le cui uscite, in ogni istante, dipendono solo dai valori degli ingressi in quello stesso istante (a meno dei ritardi, che si vedono nel modulo 2). Una rete combinatoria realizza quindi una funzione booleana, e la si può descrivere in quattro modi equivalenti: con la **tabella di verità** (la specifica), con un'**espressione** booleana (la rappresentazione matematica), con lo **schema** delle porte (la rappresentazione circuitale) e con il **diagramma temporale**, che mostra l'andamento nel tempo degli ingressi e delle uscite.

1.2 Le porte logiche

Le funzioni di una variabile sono quattro: le costanti $y = 0$ e $y = 1$, l'**identità** $y = x$, che nel circuito è un filo o un *buffer*, e la **negazione** $y = \bar{x}$ (NOT), che nei simboli si indica con un pallino.

Le funzioni di due variabili sono sedici. La tabella le elenca tutte, con i valori $f(0,0)$, $f(0,1)$, $f(1,0)$, $f(1,1)$ nella prima colonna.

$f(00)\, f(01)\, f(10)\, f(11)$	**Espressione**	**Nome**
0 0 0 0	0	costante 0
0 0 0 1	$x_1 x_2$	AND
0 0 1 0	$x_1 \bar{x}_2$	–
0 0 1 1	x_1	identità di x_1
0 1 0 0	$\bar{x}_1 x_2$	–
0 1 0 1	x_2	identità di x_2
0 1 1 0	$x_1 \oplus x_2$	XOR
0 1 1 1	$x_1 + x_2$	OR
1 0 0 0	$\overline{x_1 + x_2}$	NOR
1 0 0 1	$\overline{x_1 \oplus x_2}$	XNOR

$f(00)\,f(01)\,f(10)\,f(11)$	**Espressione**	**Nome**
1 0 1 0	$\bar{x}_2$	NOT x_2
1 0 1 1	$x_1 + \bar{x}_2$	–
1 1 0 0	$\bar{x}_1$	NOT x_1
1 1 0 1	$\bar{x}_1 + x_2$	–
1 1 1 0	$\overline{x_1 x_2}$	NAND
1 1 1 1	1	costante 1

Le funzioni con un nome sono quelle che si realizzano con una sola porta (figura 1.1). L'**AND** vale 1 solo se entrambi gli ingressi valgono 1; l'**OR** vale 1 se almeno uno vale 1; lo **XOR** (*or esclusivo*) vale 1 se gli ingressi sono diversi, cioè se esattamente uno dei due vale 1. NAND, NOR e XNOR sono le loro negazioni. Le porte AND, OR, XOR e le loro negate si estendono a più ingressi: l'AND a n ingressi vale 1 se tutti valgono 1, l'OR se almeno uno vale 1, lo XOR se il numero di ingressi a 1 è dispari.

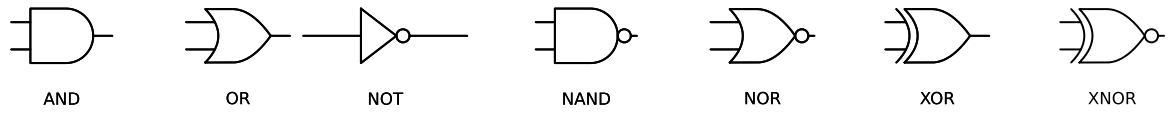


Figura 1.1

Lo XOR risolve subito la specifica "w vale 1 se uno solo dei due ingressi vale 1": la tabella è proprio quella di $w = x \oplus y$. Con tre ingressi la stessa specifica ("w vale 1 se uno solo fra x, y, t vale 1") non è più uno XOR, perché lo XOR a tre ingressi vale 1 anche quando tutti e tre sono a 1. Per scrivere un'espressione a partire da una tabella qualsiasi servono le forme canoniche del paragrafo 1.4.

1.3 Algebra di Boole

Le espressioni booleane si manipolano con regole simili a quelle dell'algebra dei numeri, ma non identiche. Ogni identità si dimostra per **induzione perfetta**: si controlla sulla tabella di verità che i due membri valgano lo stesso per tutte le combinazioni delle variabili, che sono in numero finito.

Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-01)

Proprietà dell'algebra di Boole. Ogni riga è scritta nella forma per l'OR e nella forma duale per l'AND.

Proprietà	Forma OR	Forma AND
Elemento neutro	$A + 0 = A$	$A \cdot 1 = A$
Elemento nullo	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotenza	$A + A = A$	$A \cdot A = A$
Complemento	$A + \bar{A} = 1$	$A \cdot \bar{A} = 0$
Involuzione	$\bar{\bar{A}} = A$	$\bar{\bar{A}} = A$
Commutativa	$A + B = B + A$	$AB = BA$
Associativa	$(A + B) + C = A + (B + C)$	$(AB)C = A(BC)$
Distributiva	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Assorbimento	$A + AB = A$	$A(A + B) = A$
Assorbimento del complemento	$A + \bar{A}B = A + B$	$A(\bar{A} + B) = AB$
De Morgan	$\overline{A + B} = \bar{A}\,\bar{B}$	$\overline{AB} = \bar{A} + \bar{B}$
Consenso	$AB + \bar{A}C + BC = AB + \bar{A}C$	$(A + B)(\bar{A} + C)(B + C) = (A + B)(\bar{A} + C)$

Due proprietà non hanno corrispondente nell'algebra dei numeri: l'idempotenza e la distributiva della somma rispetto al prodotto, $A + BC = (A + B)(A + C)$.

Principio di dualità. Se un'identità è vera, resta vera scambiando ovunque $+$ con $\cdot$ e 0 con 1 (le variabili e le negazioni restano). Per questo ogni proprietà della tabella compare in coppia, e i teoremi di questo modulo hanno sempre una versione AND-OR e una OR-AND.

Le leggi di **De Morgan** valgono con un numero qualsiasi di variabili: la negazione di una somma è il prodotto delle negazioni, e la negazione di un prodotto è la somma delle negazioni. Sono lo strumento per portare le negazioni dentro le parentesi e, nel paragrafo 1.7, per passare da porte AND e OR a sole NAND o NOR.

Il **termine di consenso** BC della proprietà omonima si ottiene dai due termini AB e $\bar{A}C$ che contengono una variabile in forma diretta e negata, togliendola: è ridondante, perché quando vale 1 vale 1 anche uno degli altri due. Nel modulo 2 si vedrà che, nel circuito reale, proprio questo termine ridondante può servire.

Due semplificazioni

Sia $Y = \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + AB\bar{C}$. Per l'idempotenza si può duplicare il termine centrale, poi si raccoglie:

$$Y = \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}B\bar{C} + AB\bar{C} = \bar{A}\bar{C}(\bar{B} + B) + B\bar{C}(\bar{A} + A) = \bar{A}\bar{C} + B\bar{C} = \bar{C}(\bar{A} + B).$$

Sia ora $Y = \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + \overline{AB\bar{C}}$. Con De Morgan l'ultimo termine diventa $\bar{A} + \bar{B} + C$; i primi due si raccolgono in $\bar{A}\bar{C}$, che è assorbito da $\bar{A}$:

$$Y = \bar{A}\bar{C} + \bar{A} + \bar{B} + C = \bar{A}(\bar{C} + 1) + \bar{B} + C = \bar{A} + \bar{B} + C.$$

La funzione vale 0 solo per $ABC = 110$, come si vede dalla tabella.

1.4 Forme canoniche e teorema di Shannon

Il teorema di Shannon dice come scrivere un'espressione per una funzione qualsiasi a partire dalla sua tabella.

Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-01)

Mintermini e maxtermini. Un **mintermine** di n variabili è un prodotto in cui compaiono tutte le variabili, ciascuna in forma diretta o negata: vale 1 per una sola combinazione degli ingressi. Per $ABC = 101$ il mintermine è $A\bar{B}C$ (la variabile è negata dove vale 0). Un **maxtermine** è una somma in cui compaiono tutte le variabili: vale 0 per una sola combinazione. Per $ABC = 101$ il maxtermine è $\bar{A} + B + \bar{C}$ (la variabile è negata dove vale 1).

Teorema di espansione di Shannon

Per ogni funzione booleana f e ogni sua variabile x_1:

$$f(x_1, x_2, \dots, x_n) = \bar{x}_1 \cdot f(0, x_2, \dots, x_n) + x_1 \cdot f(1, x_2, \dots, x_n)$$

e, nella forma duale,

$$f(x_1, x_2, \dots, x_n) = \big(x_1 + f(0, x_2, \dots, x_n)\big) \cdot \big(\bar{x}_1 + f(1, x_2, \dots, x_n)\big).$$

Applicando il teorema a tutte le variabili si ottengono le due **forme canoniche**: f è la somma dei mintermini delle combinazioni in cui vale 1 (**prima forma**, somma di prodotti, SOP) e il prodotto dei maxtermini delle combinazioni in cui vale 0 (**seconda forma**, prodotto di somme, POS).

La dimostrazione è immediata: per $x_1 = 0$ il membro destro della prima forma vale $f(0, x_2, \dots)$, per $x_1 = 1$ vale $f(1, x_2, \dots)$. Per due variabili le due forme sono

$$f(x_1, x_2) = \bar{x}_1\bar{x}_2\, f(0,0) + \bar{x}_1 x_2\, f(0,1) + x_1\bar{x}_2\, f(1,0) + x_1 x_2\, f(1,1)$$

$$f(x_1, x_2) = \big(x_1 + x_2 + f(0,0)\big)\big(x_1 + \bar{x}_2 + f(0,1)\big)\big(\bar{x}_1 + x_2 + f(1,0)\big)\big(\bar{x}_1 + \bar{x}_2 + f(1,1)\big).$$

Nella prima i termini con $f = 0$ spariscono, e restano i mintermini degli 1; nella seconda spariscono i fattori con $f = 1$, e restano i maxtermini degli 0.

Le due forme di una funzione di tre variabili

Sia y la funzione che vale 1 per $ABC = 000, 001, 100, 110$ e 0 altrove. La prima forma prende gli 1:

$$y = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + A\bar{B}\bar{C} + AB\bar{C} = \bar{A}\bar{B}(\bar{C} + C) + A\bar{C}(\bar{B} + B) = \bar{A}\bar{B} + A\bar{C}.$$

La seconda prende gli 0, cioè $010, 011, 101, 111$:

$$y = (A + \bar{B} + C)(A + \bar{B} + \bar{C})(\bar{A} + B + \bar{C})(\bar{A} + \bar{B} + \bar{C}).$$

Posto $P = A + \bar{B}$, i primi due fattori danno $(P + C)(P + \bar{C}) = P + C\bar{C} = P$ per la distributiva; allo stesso modo gli ultimi due danno $\bar{A} + \bar{C}$. Quindi $y = (A + \bar{B})(\bar{A} + \bar{C})$.

La forma SOP si realizza con una **rete a due livelli** AND-OR: un livello di porte AND, una per ogni prodotto, e una porta OR che li somma. La forma POS dà la rete OR-AND. Nel contare i livelli non si contano i NOT sugli ingressi, perché si suppone di avere ogni variabile disponibile sia diretta sia negata. Una rete a due livelli è la più veloce possibile, perché ogni segnale attraversa al più due porte; le forme canoniche però hanno in genere molti termini, e il problema della **sintesi** è trovare un'espressione equivalente con meno porte e meno ingressi. Non sempre esiste una sola sintesi minima: possono essercene diverse con lo stesso costo.

1.5 Mappe di Karnaugh

La semplificazione algebrica si basa sempre sullo stesso passo, $XA + X\bar{A} = X$: due prodotti che differiscono per una sola variabile si fondono in uno. La **mappa di Karnaugh** è una tabella di verità disposta in modo che questi prodotti si vedano a colpo d'occhio.

Mappa di Karnaugh

La mappa di una funzione di n variabili ha 2^n celle, una per combinazione degli ingressi. Righe e colonne sono ordinate secondo il **codice Gray** (00, 01, 11, 10), in cui due valori consecutivi differiscono per un solo bit. Due celle sono **adiacenti** quando differiscono per una sola variabile: sono adiacenti le celle vicine, e anche le celle ai bordi opposti della stessa riga o colonna, perché la mappa si richiude su se stessa.

Le combinazioni di due variabili sono i quattro vertici di un quadrato, quelle di tre gli otto vertici di un cubo, quelle di quattro i vertici di un ipercubo: due vertici sono collegati da uno spigolo se differiscono per una variabile. La mappa è il modo di disegnare questi oggetti su un foglio senza perdere le adiacenze. Oltre le quattro variabili le adiacenze non si vedono più bene, e si usano metodi tabellari o programmi.

Un **gruppo** di 2^k celle adiacenti a forma di rettangolo (anche spezzato dai bordi) che contengono tutte 1 corrisponde a un prodotto con $n-k$ variabili: restano solo quelle che hanno lo stesso valore in tutte le celle del gruppo. Una cella isolata è un mintermine, una coppia toglie una variabile, una quaterna due, e così via.

Regole di raggruppamento (sintesi SOP)

1. Coprire tutti gli 1 della mappa.
2. Usare gruppi più grandi possibile, di 2^k celle.
3. Usare il minimo numero di gruppi.

I gruppi possono sovrapporsi. Per la sintesi POS si raggruppano gli 0: ogni gruppo dà una somma in cui una variabile compare negata se nel gruppo vale 1, diretta se vale 0.

Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-02)

Implicanti primi ed essenziali. Un gruppo di 1 è un **implicante** della funzione: quando il suo prodotto vale 1, vale 1 anche la funzione. Un implicante è **primo** se non è contenuto in un gruppo più grande; è **essenziale** se copre almeno un 1 che nessun altro implicante primo copre. Una sintesi minima usa solo implicanti primi e contiene sempre tutti quelli essenziali. Il procedimento è quindi: individuare gli implicanti primi, prendere gli essenziali, e coprire gli 1 rimasti con il minor numero di implicanti primi. Quando nell'ultimo passo ci sono più scelte equivalenti, ci sono più sintesi minime.

Nelle mappe di questo modulo le righe e le colonne portano in testa le variabili, nell'ordine di Gray. Negli esempi seguenti, con tre variabili, le righe sono A e le colonne BC.

Una mappa a tre variabili, con un errore da evitare

La funzione vale 1 per $ABC = 000, 001, 010, 100, 101$.

$A\backslash BC$	**00**	**01**	**11**	**10**
0	1	1	0	1
1	1	1	0	0

Le prime due colonne formano una quaterna, in cui solo B resta costante a 0: il prodotto è $\bar{B}$. Resta l'1 in 010, che si raggruppa con la cella 000, adiacente perché la riga si richiude: il gruppo è $\bar{A}\bar{C}$. Quindi $U = \bar{B} + \bar{A}\bar{C}$.

Sarebbe sbagliato prendere tutta la riga $A = 0$ come gruppo $\bar{A}$: la cella 011 vale 0.

Una sintesi senza quaterne

La funzione vale 1 per $ABC = 001, 010, 100, 110$.

$A\backslash BC$	**00**	**01**	**11**	**10**
0	0	1	0	1
1	1	0	0	1

L'1 in 001 non ha vicini a 1: resta il mintermine $\bar{A}\bar{B}C$. Le celle 100 e 110 sono adiacenti (ai due bordi della riga) e danno $A\bar{C}$; le celle 010 e 110 sono nella stessa colonna e danno $B\bar{C}$. La sintesi minima è $U = \bar{A}\bar{B}C + A\bar{C} + B\bar{C}$, con tre implicanti tutti essenziali.

Sintesi AND-OR e OR-AND della stessa funzione

Sia $U = A \oplus B + BC$. Gli 1 sono in $010, 011, 100, 101, 111$.

$A\backslash BC$	**00**	**01**	**11**	**10**
0	0	0	1	1
1	1	1	1	0

Raggruppando gli 1 si trovano $\bar{A}B$ (le ultime due celle della prima riga), $A\bar{B}$ (le prime due della seconda) e, per l'1 in 111, a scelta BC oppure AC: due sintesi minime, $U = A\bar{B} + \bar{A}B + BC$ e $U = A\bar{B} + \bar{A}B + AC$. Raggruppando gli 0 (000, 001, 110) si trovano $\bar{A}\bar{B}$, cioè il fattore $(A + B)$, e la cella 110, cioè $(\bar{A} + \bar{B} + C)$: $U = (A + B)(\bar{A} + \bar{B} + C)$.

Con quattro variabili le righe sono AB e le colonne CD, entrambe in ordine di Gray, e si richiudono sia le righe sia le colonne: i quattro angoli sono adiacenti fra loro.

Una mappa a quattro variabili

La funzione vale 1 per i mintermini 0, 1, 2, 5, 8, 9, 10 (numerati in binario come $ABCD$).

$AB\backslash CD$	**00**	**01**	**11**	**10**
00	1	1	0	1
01	0	1	0	0
11	0	0	0	0
10	1	1	0	1

Le prime due colonne nelle righe 00 e 10 (adiacenti, perché le righe si richiudono) formano la quaterna $\bar{B}\bar{C}$. I quattro angoli formano la quaterna $\bar{B}\bar{D}$. Resta l'1 in 0101, che si raggruppa con 0001: $\bar{A}\bar{C}D$. Tutti e tre gli implicanti sono essenziali e $U = \bar{B}\bar{C} + \bar{B}\bar{D} + \bar{A}\bar{C}D$.

Ci sono funzioni che le mappe non semplificano affatto. La funzione di tre variabili che vale 1 quando il numero di ingressi a 1 è pari ha la mappa a scacchiera:

$x\backslash yz$	**00**	**01**	**11**	**10**
0	1	0	1	0
1	0	1	0	1

Nessuna cella ha vicini con lo stesso valore, quindi sia la SOP sia la POS sono le forme canoniche, con quattro termini da tre variabili ciascuno. Raggruppando gli 0 si ottiene

$$U = (x + y + \bar{z})(x + \bar{y} + z)(\bar{x} + y + z)(\bar{x} + \bar{y} + \bar{z}).$$

È la funzione $U = \overline{x \oplus y \oplus z}$, che con le porte XOR richiede due sole porte. Le funzioni di parità sono il caso peggiore per le reti a due livelli e il caso migliore per le porte XOR.

1.6 Indifferenze e variabili riportate

Spesso alcune combinazioni degli ingressi non si presentano mai, oppure il valore dell'uscita in corrispondenza di esse non interessa. In quelle celle si scrive un trattino, che si chiama **indifferenza** (*don't care*): nella sintesi lo si considera 1 se aiuta a fare gruppi più grandi, 0 altrimenti. Un'indifferenza non va mai coperta per forza.

Il trattino si usa anche negli ingressi di una tabella: una riga come "$S_1S_0 = 1\,1$, $A = -$" vuol dire che il valore di A non conta, e la riga vale per entrambi i valori. Le due cose sono diverse: la prima è un'indifferenza sull'uscita, la seconda sugli ingressi.

Sintesi SOP e POS con le indifferenze

Righe AC, colonne BD (stesso ordine di Gray):

$AC \backslash BD$	**00**	**01**	**11**	**10**
00	1	1	1	1
01	0	–	0	0
11	0	1	0	0
10	1	–	0	0

Per la SOP: la prima riga è il gruppo $\bar{A}\bar{C}$; la colonna $BD = 01$, con le due indifferenze, è il gruppo $\bar{B}D$; le righe 00 e 10 nelle colonne 00 e 01 danno $\bar{B}\bar{C}$. Quindi $U = \bar{A}\bar{C} + \bar{B}\bar{C} + \bar{B}D$.

Per la POS si raggruppano gli 0: le colonne 11 e 10 nelle righe 11 e 10 ($A = 1$, $B = 1$) danno $(\bar{A} + \bar{B})$; le colonne 00 e 10 nelle righe 01 e 11 ($C = 1$, $D = 0$) danno $(\bar{C} + D)$; le colonne 11 e 10 nelle righe 01 e 11 ($B = 1$, $C = 1$) danno $(\bar{B} + \bar{C})$. Quindi $U = (\bar{A} + \bar{B})(\bar{C} + D)(\bar{B} + \bar{C})$.

Nella sintesi OR-AND è facile sbagliare la polarità: il fattore del gruppo con $B = 1$ e $C = 1$ è $(\bar{B} + \bar{C})$, non $(\bar{B} + C)$. Conviene sempre controllare la sintesi su una cella a 1: qui $\bar{A}B\bar{C}D$ deve dare 1, e con $(\bar{B} + C)$ darebbe 0.

Una mappa può contenere, oltre a 0, 1 e trattini, anche delle **variabili riportate** (*map-entered variables*): una cella che contiene I vuol dire che in quella combinazione l'uscita vale I. Così una funzione di tre variabili si scrive su una mappa di due, e una di cinque o sei su una mappa di quattro. Le variabili riportate nascono in modo naturale con i multiplexer (modulo 2), dove ogni cella corrisponde a un ingresso dati.

Sintesi con le variabili riportate

1. Si individuano le variabili riportate.
2. Si fa la mappa con tutte le variabili riportate poste a 0 e la si sintetizza.
3. Per ogni variabile riportata si fa una mappa in cui le celle con quella variabile valgono 1, le celle con 1 diventano indifferenze (sono già coperte dalla mappa del punto 2), le celle con 0 restano 0, e le celle con le altre variabili riportate valgono 0.
4. Si sommano le sintesi: quella del punto 2 e, per ogni variabile riportata, la sua sintesi moltiplicata (AND) per la variabile.

Il risultato è una sintesi **quasi minima**: è corretta, ma non è garantito che sia la più piccola possibile, perché le variabili riportate non si raggruppano con le adiacenze delle altre.

Una variabile riportata

Righe A, colonne B: $f(0,0)=1$, $f(0,1)=I$, $f(1,0)=1$, $f(1,1)=0$.

Con $I=0$ la mappa ha gli 1 nella colonna $B=0$: $\bar{B}$. Nella mappa di I le due celle con 1 diventano indifferenze e la cella $\bar{A}B$ vale 1: con l'indifferenza in $\bar{A}\bar{B}$ si forma il gruppo $\bar{A}$. Quindi $U=\bar{B}+I\bar{A}$.

Sarebbe inutile scrivere $\bar{I}(\dots)+I(\dots)$ con due mappe complete: le celle a 1 verrebbero coperte due volte. Le celle a 1 sono già nella mappa del punto 2, e la mappa di I deve coprire solo le celle con I.

Due variabili riportate

Righe A, colonne BC:

$A\backslash BC$	**00**	**01**	**11**	**10**
0	1	1	y	y
1	1	x	0	0

Con $x=y=0$ si trovano $\bar{A}\bar{B}$ e $\bar{B}\bar{C}$. Nella mappa di x la cella 101 vale 1 e, con le indifferenze delle prime due colonne, forma la quaterna $\bar{B}$. Nella mappa di y le celle 011 e 010 valgono 1 e, con le indifferenze, formano la riga $\bar{A}$. Quindi

$$U=\bar{A}\bar{B}+\bar{B}\bar{C}+x\bar{B}+y\bar{A}.$$

1.7 Reti con sole porte NAND o NOR

Con le porte AND, OR e NOT si realizza qualsiasi funzione, perché esistono le forme canoniche. Lo stesso vale per la sola porta NAND, che quindi è **funzionalmente completa**: un NOT è una NAND con gli ingressi uniti ($\overline{A\cdot A}=\bar{A}$), un AND è una NAND seguita da un NOT, e un OR è una NAND con gli ingressi negati, perché per De Morgan $\overline{\bar{A}\,\bar{B}}=A+B$ (figura 1.2). Per dualità anche la sola porta NOR è completa.

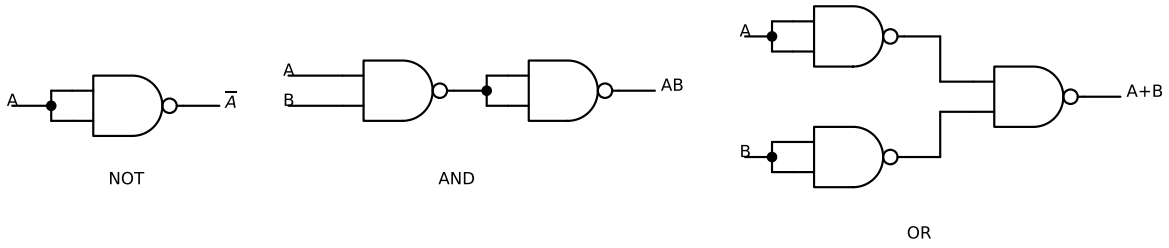


Figura 1.2

Una rete AND-OR si trasforma in una rete di sole NAND senza aggiungere livelli. Negando due volte la SOP e applicando De Morgan alla negazione esterna:

$$U=AB+CD=\overline{\overline{AB+CD}}=\overline{\overline{AB}\cdot\overline{CD}}.$$

Le porte AND del primo livello diventano NAND, e la OR del secondo livello diventa una NAND. Allo stesso modo una rete OR-AND diventa una rete di sole NOR. Le porte NAND e NOR sono le più semplici da costruire con i transistori, e questa trasformazione è quella che si usa in pratica.

Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-02)

Da POS a sole NOR. Per dualità, $U=(A+B)(C+D)=\overline{\overline{(A+B)(C+D)}}=\overline{\overline{A+B}+\overline{C+D}}$: le OR del primo livello diventano NOR e la AND del secondo livello diventa una NOR.

Esempi ed esercizi svolti

Esercizio 1. Una rete realizza $F = ACD + \bar{C}\big(\bar{A}D + A(B + D)\big)$, con 5 livelli di porte. Trovare una rete equivalente a due livelli.

Si distribuisce A e si raccoglie D fra i termini con $\bar{A}D$ e AD:

$$\bar{A}D + AB + AD = D(\bar{A} + A) + AB = D + AB,$$

quindi

$$F = ACD + \bar{C}(D + AB) = ACD + \bar{C}D + AB\bar{C} = D(AC + \bar{C}) + AB\bar{C} = D(A + \bar{C}) + AB\bar{C},$$

dove nell'ultimo passaggio si è usato l'assorbimento del complemento, $\bar{C} + AC = \bar{C} + A$. Il risultato è

$$F = AD + \bar{C}D + AB\bar{C},$$

una rete AND-OR a due livelli con tre porte AND e una OR.

> Il termine $AB\bar{C}$ non si può perdere per strada: per $ABCD = 1100$ la rete di partenza vale 1, mentre $AD + \bar{C}D$ vale 0.

Esercizio 2. Sintesi SOP e POS della funzione con la mappa (righe X, colonne YZ)

$X \backslash YZ$	**00**	**01**	**11**	**10**
0	1	0	1	0
1	1	0	0	1

Per la SOP: la colonna 00 dà $\bar{Y}\bar{Z}$; le celle 100 e 110 (adiacenti ai bordi) danno $X\bar{Z}$; l'1 in 011 resta solo: $U = \bar{Y}\bar{Z} + X\bar{Z} + \bar{X}YZ$. Per la POS: la colonna 01 dà $(Y + \bar{Z})$; le celle 101 e 111 danno $(\bar{X} + \bar{Z})$; lo 0 in 010 resta solo e dà $(X + \bar{Y} + Z)$. Quindi $U = (Y + \bar{Z})(\bar{X} + \bar{Z})(X + \bar{Y} + Z)$.

Esercizio 3. Semplificare $x + xy + xyz + xyzw + xz + xw + xzw$.

Raccogliendo x: $x(1 + y + yz + yzw + z + w + zw) = x \cdot 1 = x$, perché una somma che contiene 1 vale 1. È l'assorbimento, $x + xP = x$, applicato a ogni termine.

Riepilogo

Una funzione booleana di n variabili è descritta dalla sua tabella di verità, che ha 2^n righe; le funzioni possibili sono 2^{2^n}. Le funzioni di una e due variabili sono realizzate dalle porte NOT, AND, OR, XOR e dalle loro negate. L'algebra di Boole permette di trasformare un'espressione in un'altra equivalente; le proprietà vengono a coppie per il principio di dualità, e le leggi di De Morgan scambiano somme e prodotti sotto negazione. Il teorema di Shannon dà le due forme canoniche: la somma dei mintermini degli 1 e il prodotto dei maxtermini degli 0, da cui le reti a due livelli AND-OR e OR-AND. Le mappe di Karnaugh rendono visibili le adiacenze: si coprono gli 1 (o gli 0) con il minimo numero di gruppi rettangolari di 2^k celle, i più grandi possibile, usando gli implicanti primi essenziali. Le indifferenze si usano solo se allargano i gruppi; le variabili riportate permettono di sintetizzare con mappe più piccole una funzione di più variabili, con un risultato quasi minimo. Le porte NAND e NOR sono complete da sole, e una rete AND-OR diventa una rete di sole NAND con la stessa struttura.

2 · Reti combinatorie standard, ritardi e aritmetica

In questo modulo. Si usano le forme canoniche, le mappe di Karnaugh e le variabili riportate del modulo 1 (*Algebra booleana e sintesi delle reti combinatorie*). Si studiano le reti combinatorie che si usano come blocchi già pronti: multiplexer, demultiplexer, decoder, encoder. Poi si guarda che cosa succede nel circuito reale, dove ogni porta ha un ritardo, e come i ritardi producano le alee. L'ultima parte è l'aritmetica binaria: la rappresentazione dei numeri e i codici, la somma, il complemento a 2 per i numeri negativi, le reti che sommano, sottraggono, confrontano e moltiplicano, fino all'unità aritmetico-logica. Alla fine del modulo si sa progettare con i blocchi standard, riconoscere ed eliminare un'alea statica, e fare a mano e con una rete le operazioni sui numeri binari con e senza segno.

2.1 Multiplexer

Il **multiplexer** (MUX) è un selettore: ha alcuni ingressi dati, alcuni ingressi di selezione e un'uscita, che copia l'ingresso dati scelto dalla selezione. Il più semplice è il MUX 2→1, con ingressi dati S_0 e S_1, selezione SEL e uscita U. La specifica a parole è: $U = S_0$ se $SEL = 0$, $U = S_1$ se $SEL = 1$.

SEL	S_1	S_0	U
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

La prima forma canonica ha quattro mintermini, che si raccolgono a due a due:

$$U = \overline{SEL}\,\bar{S}_1 S_0 + \overline{SEL}\,S_1 S_0 + SEL\,S_1\bar{S}_0 + SEL\,S_1 S_0 = \overline{SEL}\,S_0 + SEL\,S_1.$$

La seconda ha quattro maxtermini, che si raccolgono allo stesso modo: posto $T = SEL + S_0$, i due fattori $(T + S_1)(T + \bar{S}_1)$ danno T, e lo stesso vale per gli altri due. Quindi $U = (SEL + S_0)(\overline{SEL} + S_1)$. Con la mappa le due sintesi si trovano subito: righe SEL, colonne S_1S_0.

$SEL \backslash S_1S_0$	**00**	**01**	**11**	**10**
0	0	1	1	0
1	0	0	1	1

Il circuito della SOP ha due AND e una OR (figura 2.1): SEL entra diretto in una AND e negato nell'altra, quindi una sola delle due AND è "aperta" e lascia passare il suo ingresso dati.

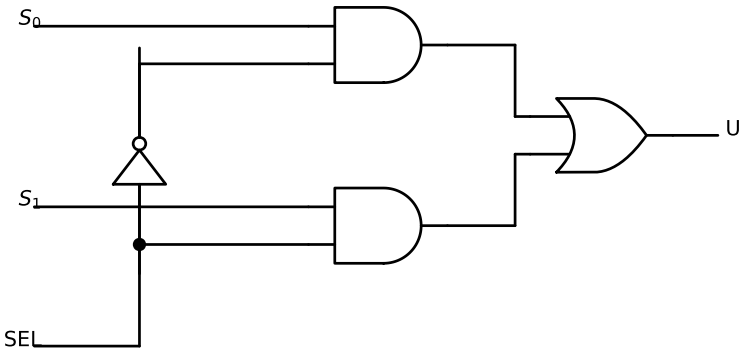


Figura 2.1

Il MUX 4→1 ha quattro ingressi dati $I_0, \dots, I_3$ e due di selezione $S_1 S_0$, che formano il numero binario dell'ingresso scelto:

S_1	S_0	U
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$$U = \bar{S}_1 \bar{S}_0 \, I_0 + \bar{S}_1 S_0 \, I_1 + S_1 \bar{S}_0 \, I_2 + S_1 S_0 \, I_3 .$$

In generale un MUX $2^n \to 1$ ha n ingressi di selezione, e la sua uscita è la somma dei prodotti fra ogni mintermine delle selezioni e l'ingresso dati corrispondente. Un MUX grande si costruisce con MUX più piccoli: con tre MUX 2→1 si ottiene un MUX 4→1, facendo scegliere ai primi due con S_0 fra I_0, I_1 e fra I_2, I_3, e all'ultimo con S_1 fra le loro uscite (figura 2.2).

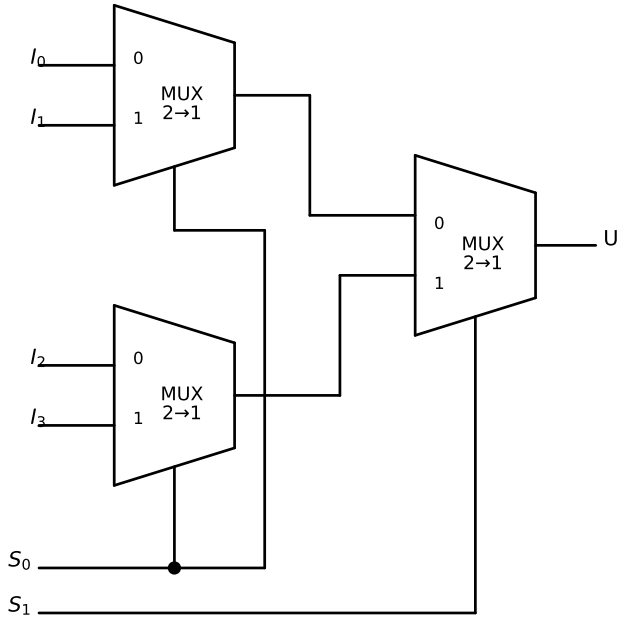


Figura 2.2

Il MUX è una **rete universale**. Se si collegano le n variabili di una funzione agli ingressi di selezione di un MUX $2^n \to 1$ e si mette su ogni ingresso dati la costante 0 o 1 della riga corrispondente della tabella di verità, l'uscita realizza la funzione. Per esempio lo XOR $A \oplus B$ si ottiene da un MUX 4→1 con A su S_1, B su S_0 e ingressi dati $0, 1, 1, 0$ (figura 2.3).

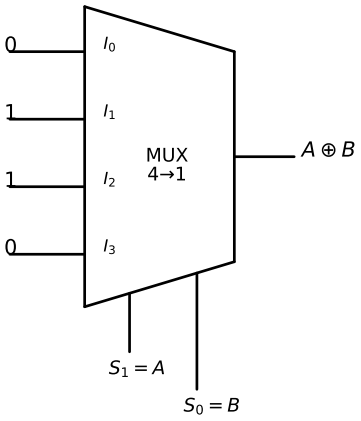


Figura 2.3

Il MUX 4→1 si può anche leggere come una mappa a due variabili (S_1, S_0) con gli ingressi dati nelle celle: è una mappa a variabili riportate. Di qui il legame con il paragrafo 1.6: una funzione di n variabili si realizza anche con

un MUX $2^{n-1} \to 1$, mettendo $n-1$ variabili sulle selezioni e scrivendo negli ingressi dati 0, 1, la variabile rimasta o la sua negazione.

2.2 Demultiplexer, decoder, encoder

Il **demultiplexer** (DEMUX) fa l'operazione inversa del MUX: ha un ingresso dati IN, n ingressi di selezione e 2^n uscite, e porta IN sull'uscita scelta, mentre le altre restano a 0. Per il DEMUX 1→4:

$$U_0 = \bar{S}_1 \bar{S}_0 \, IN, \quad U_1 = \bar{S}_1 S_0 \, IN, \quad U_2 = S_1 \bar{S}_0 \, IN, \quad U_3 = S_1 S_0 \, IN.$$

S_1	S_0	U_0	U_1	U_2	U_3
0	0	IN	0	0	0
0	1	0	IN	0	0
1	0	0	0	IN	0
1	1	0	0	0	IN

Il **decoder** $n \to 2^n$ ha n ingressi e 2^n uscite, e per ogni combinazione degli ingressi porta a 1 una e una sola uscita: l'uscita U_k vale 1 quando gli ingressi formano il numero k. Ogni uscita è quindi un mintermine degli ingressi: nel decoder 3→8, $U_0 = \bar{I}_2 \bar{I}_1 \bar{I}_0$, ..., $U_7 = I_2 I_1 I_0$. Un DEMUX è un decoder con un ingresso di abilitazione: se $IN = 0$ tutte le uscite sono a 0, se $IN = 1$ funziona come decoder. Poiché un decoder produce tutti i mintermini, un decoder seguito da una OR degli opportuni mintermini realizza qualsiasi funzione nella prima forma canonica.

> **Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-02)**
>
> **Encoder.** L'**encoder** $2^n \to n$ fa l'inverso del decoder: ha 2^n ingressi, di cui se ne suppone attivo uno alla volta, e dà in uscita il numero binario dell'ingresso attivo. Se più ingressi possono essere attivi insieme si usa l'**encoder con priorità**, che codifica quello di indice più alto, con un'uscita V che dice se almeno un ingresso è attivo. Per 4 ingressi:
>
> | I_3 | I_2 | I_1 | I_0 | Y_1 | Y_0 | V |
> |---|---|---|---|---|---|---|
> | 0 | 0 | 0 | 0 | – | – | 0 |
> | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
> | 0 | 0 | 1 | – | 0 | 1 | 1 |
> | 0 | 1 | – | – | 1 | 0 | 1 |
> | 1 | – | – | – | 1 | 1 | 1 |
>
> I trattini negli ingressi sono indifferenze sugli ingressi (paragrafo 1.6). Dalla tabella: $Y_1 = I_3 + I_2$, $Y_0 = I_3 + \bar{I}_2 I_1$, $V = I_3 + I_2 + I_1 + I_0$.

Una porta logica programmabile

Una rete ha due ingressi di controllo S_0, S_1 e due ingressi dati A, B, e deve comportarsi come una porta diversa a seconda del controllo:

S_0	S_1	U
0	0	$A \cdot B$
0	1	$A + B$
1	0	$\bar{A}$
1	1	–

La combinazione 11 non si usa: è un'indifferenza su tutta la riga. Sulla mappa a quattro variabili (righe S_0S_1, colonne AB)

$S_0S_1 \backslash AB$	**00**	**01**	**11**	**10**
00	0	0	1	0
01	0	1	1	1
11	–	–	–	–
10	1	1	0	0

la sintesi AND-OR è $U = S_1A + S_1B + S_0\bar{A} + \bar{S}_0AB$ e quella OR-AND è $U = (\bar{S}_0 + \bar{A})(S_0 + S_1 + A)(S_0 + S_1 + B)(A + B + \bar{S}_1)$. L'ultimo fattore copre lo 0 in $S_0S_1AB = 0100$ usando l'indifferenza in 1100; si può scegliere anche $(S_0 + A + B)$, che copre lo stesso 0 insieme a quello in 0000. Le indifferenze della riga 11 permettono di scrivere S_1A invece di $\bar{S}_0S_1A$, e $S_0\bar{A}$ invece di $S_0\bar{S}_1\bar{A}$.

2.3 Ritardi di propagazione e alee

Nel modulo 1 le porte erano ideali: l'uscita cambiava nello stesso istante degli ingressi. Una porta reale impiega un tempo per rispondere, il **ritardo di propagazione** t_p, che è diverso da porta a porta e cambia con la temperatura, con il carico e con il verso della commutazione. Il diagramma temporale reale di una rete combinatoria mostra le uscite che cambiano un po' dopo gli ingressi, e una rete con più livelli accumula i ritardi di tutte le porte che un segnale attraversa.

L'esempio più semplice è $OUT = IN + \overline{IN}$, che per l'algebra vale sempre 1. Nel circuito IN arriva alla OR direttamente, $\overline{IN}$ attraverso un NOT che ritarda di t_p. Quando IN scende da 1 a 0, per un tempo pari al ritardo del NOT entrambi gli ingressi della OR sono a 0, e l'uscita fa un impulso a 0 che non dovrebbe esserci (figura 2.4).

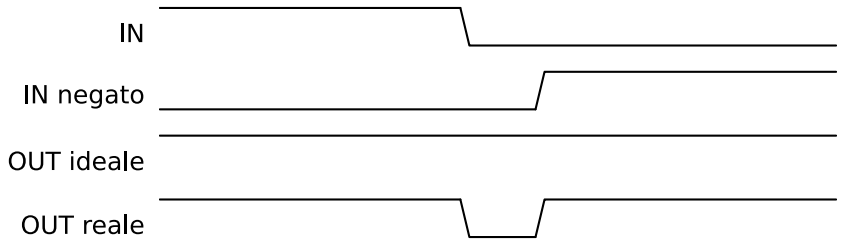


Figura 2.4

Alea

Un'**alea** (*hazard*) è una variazione transitoria e indesiderata dell'uscita di una rete combinatoria, dovuta ai diversi ritardi dei percorsi che i segnali seguono dagli ingressi all'uscita. L'alea è **statica** se l'uscita dovrebbe restare costante e invece fa un impulso (alea statica di 1 se dovrebbe restare a 1, di 0 se dovrebbe restare a 0); è **dinamica** se l'uscita dovrebbe cambiare una volta e invece oscilla prima di assestarsi.

Le alee statiche di una rete a due livelli si vedono sulla mappa. Si prenda $U = \bar{A}B + AC$, con $B = C = 1$ e A che scende da 1 a 0: la funzione passa dalla cella 111 alla cella 011 e vale 1 in entrambe. Ma le due celle sono coperte

da gruppi diversi: in 111 l'uscita è tenuta a 1 dalla AND di AC, in 011 dalla AND di $\bar{A}B$. Se la prima si spegne prima che la seconda si accenda, cosa che succede perché $\bar{A}$ passa attraverso un NOT in più, l'uscita scende a 0 per un istante.

> **Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-02)**
>
> **Il termine di consenso elimina l'alea.** Se si aggiunge alla sintesi il gruppo BC, che copre entrambe le celle, durante la transizione la sua AND resta accesa e tiene l'uscita a 1: $U = \bar{A}B + AC + BC$ è la stessa funzione (è la proprietà del consenso del paragrafo 1.3) ma non ha l'alea (figura 2.5). In generale una sintesi SOP è priva di alee statiche di 1 per le variazioni di una sola variabile se ogni coppia di celle adiacenti a 1 è coperta da almeno un gruppo comune. Si paga con porte in più: la sintesi senza alee non è più minima.

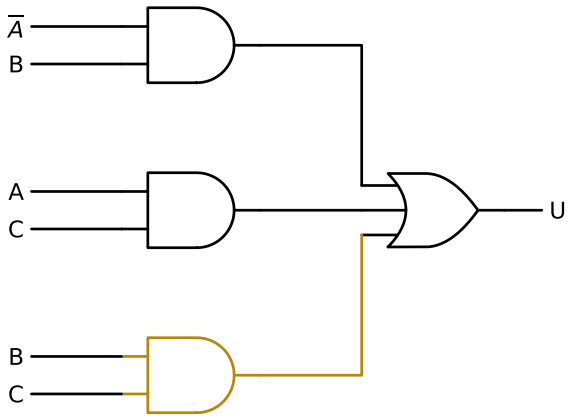


Figura 2.5

Le alee contano quando l'uscita di una rete combinatoria viene usata mentre è ancora in transizione. La soluzione più generale è non guardarla finché non si è assestata: è quello che fa il **clock** nelle reti sincrone del modulo 3, dove le uscite delle reti combinatorie si campionano solo in istanti in cui sono certamente a regime.

2.4 Numeri e codici

Un numero naturale si scrive in **notazione posizionale** in base B con le cifre $a_k \dots a_1 a_0$, ciascuna fra 0 e $B-1$, e il suo valore è

$$N = a_k B^k + a_{k-1} B^{k-1} + \cdots + a_1 B + a_0.$$

In base 10, $134 = 1 \cdot 10^2 + 3 \cdot 10 + 4$. In base 2 le cifre sono bit: $01010_2 = 1 \cdot 2^3 + 1 \cdot 2^1 = 10$. Con n bit si rappresentano i naturali da 0 a $2^n - 1$; con 4 bit da $0000 = 0$ a $1111 = 15$. Il bit di peso massimo si chiama MSB (*most significant bit*), quello di peso minimo LSB.

Da binario a decimale si sommano le potenze di 2 dei bit a 1: $1001_2 = 2^3 + 2^0 = 9$ e $10100110010_2 = 1024 + 256 + 32 + 16 + 2 = 1330$. **Da decimale a binario** si divide ripetutamente per 2: i resti, dal primo all'ultimo, sono i bit dall'LSB all'MSB. Per 38: $38 = 2 \cdot 19 + 0$, $19 = 2 \cdot 9 + 1$, $9 = 2 \cdot 4 + 1$, $4 = 2 \cdot 2 + 0$, $2 = 2 \cdot 1 + 0$, $1 = 2 \cdot 0 + 1$, quindi $38 = 100110_2$.

Le basi 8 (**ottale**) e 16 (**esadecimale**) sono comode perché una cifra corrisponde a un gruppo fisso di bit, 3 per l'ottale e 4 per l'esadecimale, e si convertono a gruppi partendo da destra. In esadecimale le cifre da 10 a 15 si scrivono A, …, F.

- $33_8 = 3 \cdot 8 + 3 = 27$, e in binario $011\,011_2$; viceversa $001\,010_2 = 12_8 = 10$.
- $0101\,1110_2 = 5E_{16} = 5 \cdot 16 + 14 = 94$; per divisioni successive per 16, $94 = 16 \cdot 5 + 14$ dà di nuovo $5E$.

Il **BCD** (*binary coded decimal*, codice 8421) scrive ogni cifra decimale con i suoi 4 bit: $1013 \rightarrow$ 0001 0000 0001 0011. È comodo per i visualizzatori, ma spreca 6 delle 16 combinazioni di 4 bit (da 1010 a 1111).

Il **codice Gray** ordina le combinazioni di n bit in modo che due consecutive differiscano per un solo bit. Si costruisce per riflessione: si scrive la sequenza a $n-1$ bit, poi la stessa al contrario, e si aggiunge davanti 0 alla

prima metà e 1 alla seconda. Da $0, 1$ si ottiene $00, 01, 11, 10$, e da questa $000, 001, 011, 010, 110, 111, 101, 100$. È l'ordine delle righe e delle colonne delle mappe di Karnaugh.

Un **bit di parità** aggiunto a un dato permette di accorgersi di un errore di trasmissione su un bit. Con la parità pari il bit vale lo XOR di tutti i bit del dato, così il numero totale di 1 è pari: per il dato 0111010, che ha quattro 1, il bit di parità è 0. Chi riceve rifà lo XOR di tutto: se viene 1, almeno un bit è cambiato. Due errori si compensano e non si vedono; per controlli più robusti si aggiunge in coda ai dati una somma di controllo (*checksum*).

2.5 Somma binaria

La somma si fa in colonna come in base 10, con il riporto: in base 10 $39 + 12 = 51$ perché $9 + 2 = 11$ dà 1 con riporto 1. In base 2, $1 + 1 = 10$: cifra 0 e riporto 1. Per esempio

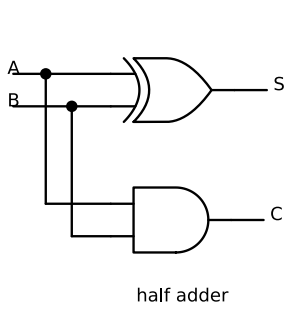
$$1001010_2 + 110110_2 = 10000000_2,$$

cioè $74 + 54 = 128$: i riporti si propagano fino all'ultima colonna e il risultato ha un bit in più degli addendi.

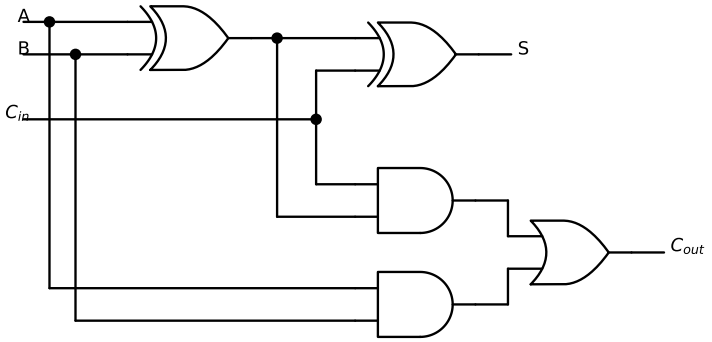
Il **semisommatore** (*half adder*) somma due bit e dà la somma S e il riporto C:

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

quindi $S = A \oplus B$ e $C = AB$. Il **sommatore completo** (*full adder*) somma due bit e il riporto in ingresso C_{in}: la somma è lo XOR a tre ingressi $S = A \oplus B \oplus C_{in}$, e il riporto vale 1 quando almeno due dei tre ingressi valgono 1, $C_{out} = AB + AC_{in} + BC_{in}$ (la sintesi della mappa ha tre coppie). La figura 2.6 mostra i due circuiti.



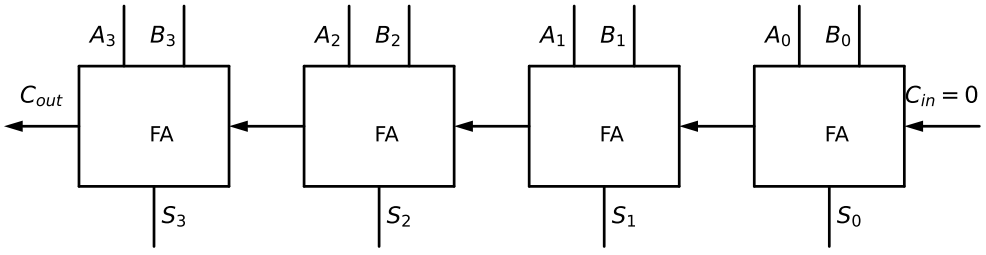
half adder



full adder

Figura 2.6

Il **sommatore a cascata** (*ripple carry adder*) a n bit mette in fila n full adder: il riporto in uscita di ogni bit è il riporto in ingresso del successivo, e il riporto in ingresso del bit 0 vale 0 (figura 2.7). Il blocco ADD4 ha ingressi $A_3 \dots A_0$, $B_3 \dots B_0$ e C_{in}, uscite $S_3 \dots S_0$ e il riporto finale; due ADD4 in cascata fanno un ADD8. Il difetto è la velocità: nel caso peggiore il riporto deve attraversare tutti gli n stadi, e il ritardo cresce con n. Per i numeri naturali, un riporto finale a 1 vuol dire che il risultato non sta in n bit: è un **overflow**.



il riporto attraversa gli stadi da destra a sinistra

Figura 2.7

2.6 Sottrazione e numeri negativi

La sottrazione in colonna usa il **prestito**: $0 - 1$ non si può fare, quindi si prende 1 dalla colonna a sinistra, e la colonna vale $10_2 - 1 = 1$. Per esempio $1010010_2 - 111111_2 = 0010011_2$, cioè $82 - 63 = 19$.

Per i numeri negativi l'idea più immediata è usare il primo bit come segno e gli altri come modulo (**modulo e segno**). Ha due difetti: lo zero ha due rappresentazioni (0000 e 1000), e il sommatore del paragrafo 2.5 non funziona più. Per esempio $7 + (-2)$ con 4 bit diventa $0111 + 1010 = 10001$, che non è 5.

> **Complemento a 2**
>
> Con n bit, nella rappresentazione in **complemento a 2** il bit più significativo ha peso -2^{n-1} invece di $+2^{n-1}$:
>
> $$N = -a_{n-1}\,2^{n-1} + a_{n-2}\,2^{n-2} + \cdots + a_1 \cdot 2 + a_0.$$
>
> I numeri rappresentabili vanno da -2^{n-1} a $2^{n-1} - 1$; quelli con il primo bit a 1 sono i negativi. Per cambiare segno a un numero si **negano tutti i bit e si somma 1**.

Con 3 bit:

bit	valore		bit	valore
000	0		100	-4
001	1		101	-3
010	2		110	-2
011	3		111	-1

Si controlla la regola: -3 si ottiene da 011 negando (100) e sommando 1 (101); -0 dà $111 + 1 = 1000$, cioè 000 sui 3 bit, e lo zero è unico. Con 4 bit i valori vanno da $1000 = -8$ a $0111 = 7$, e $1111 = -1$. Il numero di bit va sempre specificato: 1111 vale -1 su 4 bit e 15 se è un naturale, e -11 su 8 bit si ottiene da 00001011 negando (11110100) e sommando 1: 11110101.

> **Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-03)**
>
> **Estensione del segno.** Per scrivere un numero in complemento a 2 su più bit si ripete il bit di segno a sinistra: $-3 = 101$ su 3 bit è 11111101 su 8 bit, e $3 = 011$ è 00000011.

Il vantaggio del complemento a 2 è che la somma si fa con lo stesso sommatore dei naturali, ignorando il riporto finale:

- $3 + 4$: $0011 + 0100 = 0111 = 7$;
- $-7 + 3$: $1001 + 0011 = 1100 = -4$;
- $-8 + 6$: $1000 + 0110 = 1110 = -2$;
- $7 + (-2)$: $0111 + 1110 = 1\,0101$, e scartando il riporto $0101 = 5$.

> **Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-03)**
>
> **Overflow in complemento a 2.** Il risultato è sbagliato quando non sta nell'intervallo $[-2^{n-1}, 2^{n-1} - 1]$, e questo può succedere solo sommando due numeri dello stesso segno. Si riconosce perché il risultato ha il segno opposto: con 4 bit $5 + 4$ dà $0101 + 0100 = 1001$, che vale -7. Il riporto finale qui non serve: l'overflow c'è quando il riporto che entra nel bit di segno è diverso da quello che ne esce, $V = c_n \oplus c_{n-1}$.
>
> **Sottrattore.** Poiché $A - B = A + (-B) = A + \bar{B} + 1$, un sommatore diventa un sottrattore negando B e mettendo $C_{in} = 1$. Con una XOR davanti a ogni bit di B, comandata da un segnale SUB che va anche su C_{in}, la stessa rete somma ($SUB = 0$) o sottrae ($SUB = 1$).

2.7 Prodotto, confronto e unità aritmetico-logica

Il prodotto binario si fa come in base 10, ma i prodotti parziali sono solo copie del moltiplicando, spostate, o zeri: per ogni bit a 1 del moltiplicatore si somma il moltiplicando spostato della sua posizione. Per esempio $10110 \times 111 = 10110 + 101100 + 1011000 = 10011010$, cioè $22 \cdot 7 = 154$. Il prodotto di due numeri di n bit ha fino a $2n$ bit. Una rete moltiplicatrice combinatoria fa i prodotti parziali con porte AND (il prodotto di due bit è il loro AND) e li somma con una schiera di sommatori.

> **Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-03)**
>
> **Comparatore.** Il comparatore di due bit ha tre uscite: $A > B$ vale $A\bar{B}$, $A = B$ vale $\overline{A \oplus B}$, $A < B$ vale $\bar{A}B$. Per due numeri di n bit, l'uguaglianza è l'AND delle XNOR bit a bit; per il confronto di grandezza fra naturali si calcola $A - B$ con il sottrattore: c'è un prestito finale (il riporto finale del sommatore vale 0) se e solo se $A < B$.

L'**unità aritmetico-logica** (ALU, *arithmetic logic unit*) mette insieme più reti che lavorano in parallelo sugli stessi operandi A e B, per esempio un sommatore, un moltiplicatore e un comparatore, e un MUX che sceglie quale risultato mandare in uscita secondo un codice di operazione S_1S_0: 00 somma, 01 prodotto, 10 confronto (figura 2.8).

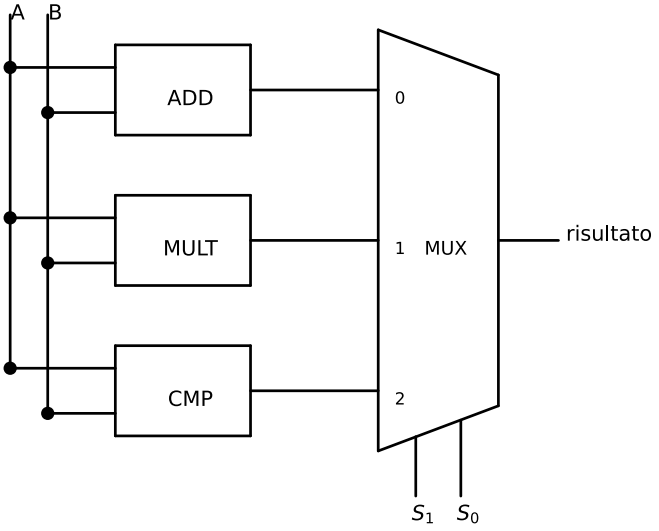


Figura 2.8

> **Virgola fissa e virgola mobile**
>
> I numeri non interi si rappresentano in **virgola fissa** decidendo quanti bit stanno dopo la virgola: con tre bit frazionari $0{,}375 = 0{,}011_2$, perché $0{,}375 = 2^{-2} + 2^{-3}$. La precisione assoluta è la stessa per tutti i numeri. In **virgola mobile** il numero si scrive come in notazione scientifica, $7{,}3 \cdot 10^2$, con un bit di segno, un esponente e una mantissa: la precisione relativa è la stessa per numeri grandi e piccoli, mentre quella assoluta cambia con l'esponente. I formati standard non fanno parte di questo corso.

Esempi ed esercizi svolti

Esercizio 1. Una rete con un MUX 4→1, una OR, una XNOR e una AND ha la tabella di verità che vale 1 solo per $ABC = 100$ e 111. Trovare le sintesi minime.

Con righe A e colonne BC, la riga $A = 0$ è tutta a 0, la riga $A = 1$ vale $1, 0, 1, 0$. Gli 1 sono isolati: $U = A\bar{B}\bar{C} + ABC$. Raggruppando gli 0, la riga $A = 0$ dà il fattore A, e i due 0 della riga $A = 1$ in 101 e 110 danno $(B + \bar{C})$ e $(\bar{B} + C)$: $U = A(B + \bar{C})(\bar{B} + C)$. Le due sintesi dicono la stessa cosa: $U = A \cdot \overline{B \oplus C}$, che è proprio la AND fra A e una XNOR.

Esercizio 2. Conversioni. $11 = 1011_2$ (resti $1, 1, 0, 1$); $10 = 12_8$ (resti $2, 1$); $30 = 1E_{16}$ (resti $14, 1$); -11 in complemento a 2 su 8 bit è 11110101.

Esercizio 3. Operazioni. $1010010 + 110010 = 10000100$ ($82 + 50 = 132$). $1010100 - 110001 = 0100011$ ($84 - 49 = 35$) e $1010100 - 111111 = 0010101$ ($84 - 63 = 21$). $1010 \times 110 = 10100 + 101000 = 111100$ ($10 \cdot 6 = 60$) e $1010 \times 10110 = 10100 + 101000 + 10100000 = 11011100$ ($10 \cdot 22 = 220$).

Esercizio 4. Generatore di parità per 4 bit $B_3B_2B_1B_0$. Il bit di parità pari è $P = B_3 \oplus B_2 \oplus B_1 \oplus B_0$, che si realizza con tre porte XOR a due ingressi, in fila oppure ad albero ($(B_3 \oplus B_2) \oplus (B_1 \oplus B_0)$), più veloce perché i livelli sono due invece di tre). Con la mappa sarebbe una scacchiera, senza nessun raggruppamento possibile.

Riepilogo

I blocchi standard delle reti combinatorie sono il multiplexer, che sceglie un ingresso dati secondo la selezione, il demultiplexer, che porta un ingresso su un'uscita scelta, il decoder, che accende una sola uscita per ogni combinazione e quindi produce tutti i mintermini, e l'encoder, che fa l'inverso. Il MUX è universale: con le variabili sulle selezioni e la tabella di verità sugli ingressi dati realizza qualsiasi funzione. Nel circuito reale ogni porta ha un ritardo di propagazione, e percorsi di lunghezza diversa producono alee: un'alea statica si vede sulla mappa come una transizione fra celle a 1 coperte da gruppi diversi, e si elimina con il termine di consenso. I numeri si scrivono in notazione posizionale; ottale ed esadecimale raggruppano i bit a 3 e a 4, il BCD codifica le cifre decimali e il codice Gray cambia un bit alla volta. La somma si fa con half e full adder in cascata; i numeri negativi si rappresentano in complemento a 2, che si ottiene negando i bit e sommando 1, e con cui la somma si fa con lo stesso sommatore. L'overflow in complemento a 2 si riconosce dal segno del risultato. Sottrattore, comparatore e moltiplicatore si costruiscono con il sommatore, e l'ALU li riunisce con un MUX che sceglie l'operazione.

3 · Flip-flop, registri e contatori

In questo modulo. Si usano l'algebra e le mappe del modulo 1 (*Algebra booleana e sintesi delle reti combinatorie*), i multiplexer, i ritardi e i sommatori del modulo 2 (*Reti combinatorie standard, ritardi e aritmetica*). Si passa dalle reti combinatorie alle **reti sequenziali**, le cui uscite dipendono anche da ciò che è successo prima. Si vede come una retroazione crei un elemento di memoria, il latch, e perché nelle reti sincrone si usino i flip-flop che cambiano solo sul fronte del clock. Poi si studiano i parametri temporali che fissano la frequenza massima di una rete, e i due blocchi costitutivi delle reti sequenziali: i registri e i contatori. Alla fine del modulo si sa scegliere e usare i flip-flop D, JK e T, calcolare il periodo minimo del clock, progettare registri e contatori sincroni e analizzare una rete sincrona disegnandone il diagramma temporale.

3.1 Dalla retroazione alla memoria

In una rete combinatoria l'uscita dipende solo dagli ingressi presenti. Se però un'uscita viene riportata in ingresso (**retroazione**), la rete può ricordare. Si prenda una OR con un ingresso A e l'altro collegato alla propria uscita U. Finché $A = 0$ e $U = 0$ la situazione è stabile; se A fa un impulso a 1, U va a 1 e, attraverso la retroazione, resta a 1 anche quando A torna a 0. La rete ha memorizzato un 1, ma non può più dimenticarlo.

Per poterlo cancellare si mette sulla retroazione una AND con un secondo ingresso: $U = A + \bar{R}\,U$. Con $R = 1$ la retroazione è interrotta e U torna a 0; con $R = 0$ e $A = 0$ l'uscita conserva il suo valore. I due comandi si chiamano **set** (S, porta l'uscita a 1) e **reset** (R, la porta a 0). C'è un punto che una rete combinatoria non ha: all'accensione, se nessuno dà un comando, il valore dell'uscita non è definito. Ogni rete sequenziale ha quindi bisogno di un comando di inizializzazione.

> **Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-04)**
>
> **Stato ed equazione caratteristica.** Il valore memorizzato si chiama **stato** e si indica con Q; il valore che assumerà dopo un comando si indica con Q^+. Per la rete appena vista, scritta con S al posto di A, l'equazione caratteristica è $Q^+ = S + \bar{R}\,Q$: lo stato successivo dipende dagli ingressi e dallo stato presente. È la differenza fra una rete sequenziale e una combinatoria.

3.2 Latch

Il **latch SR** si realizza con due porte NOR incrociate: ciascuna riceve un ingresso esterno e l'uscita dell'altra (figura 3.1). Le uscite sono Q e $\bar{Q}$, che in funzionamento normale sono una la negazione dell'altra.

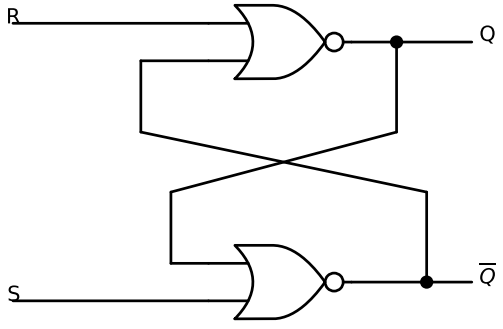


Figura 3.1

S	R	Q^+	$\bar{Q}^+$	**Comando**
0	0	Q	$\bar{Q}$	memorizzazione: conserva lo stato
0	1	0	1	reset
1	0	1	0	set
1	1	–	–	non ammesso

Questa è la **tabella caratteristica** del latch: dagli ingressi e dallo stato presente dà lo stato successivo. La combinazione $S = R = 1$ non è ammessa perché porta entrambe le uscite a 0, e $\bar{Q}$ non è più la negazione di Q. C'è anche un problema peggiore: se da $S = R = 1$ i due ingressi tornano a 0 nello stesso istante, le due NOR cercano di commutare insieme e il risultato dipende da quale è più veloce. In un circuito perfettamente simmetrico le uscite oscillerebbero; nella realtà nessun circuito è perfetto e una delle due vince, ma non si sa quale. La figura 3.2 mostra un diagramma temporale con le configurazioni stabili e questo caso finale.

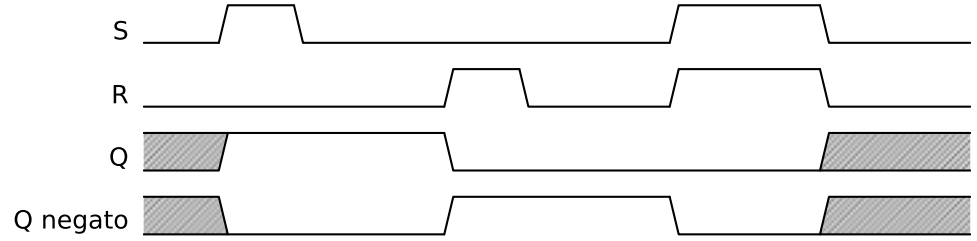


Figura 3.2

Con le leggi di De Morgan lo stesso latch si realizza con due NAND incrociate. In questo caso gli ingressi sono **attivi bassi**: il comando arriva quando valgono 0, e si indicano con $\bar{S}$ e $\bar{R}$. La combinazione vietata è $\bar{S} = \bar{R} = 0$.

Un **latch con abilitazione** aggiunge un ingresso EN: due porte davanti al latch lasciano passare S e R solo quando $EN = 1$. Con $EN = 0$ il latch conserva lo stato qualunque cosa facciano S e R; con $EN = 1$ si comporta come il latch SR. Collegando $S = D$ e $R = \bar{D}$ si ottiene il **latch D**: con $EN = 1$ l'uscita copia D, con $EN = 0$ conserva l'ultimo valore. Il latch D non ha combinazioni vietate, perché S e R non valgono mai 1 insieme (figura 3.3).

EN	D	Q^+
0	–	Q
1	0	0
1	1	1

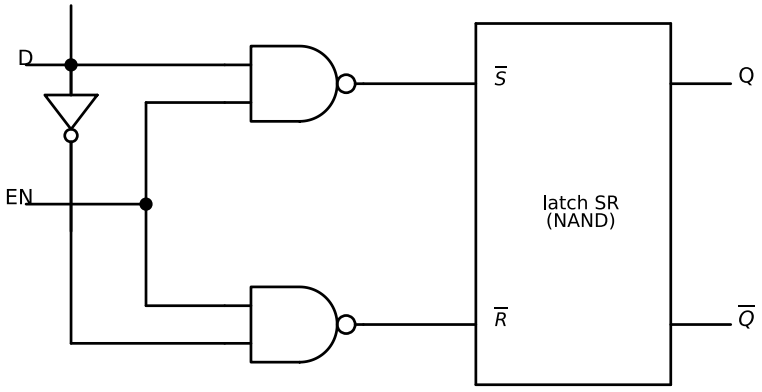


Figura 3.3

Il **latch JK** elimina la combinazione vietata in un altro modo: le due porte d'ingresso ricevono anche le uscite incrociate, così J agisce solo se $Q = 0$ e K solo se $Q = 1$.

J	K	Q^+	**Comando**
0	0	Q	memorizzazione
0	1	0	reset
1	0	1	set
1	1	$\bar{Q}$	commutazione

Con $J = K = 1$ lo stato si inverte. Ma in un latch la commutazione non si ferma: appena Q cambia, la retroazione cambia gli ingressi delle porte, e lo stato si inverte di nuovo, finché l'abilitazione resta attiva. L'uscita oscilla.

Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-04)

Trasparenza e metastabilità. Un latch abilitato è **trasparente**: ogni variazione dell'ingresso arriva subito all'uscita. In una rete in cui l'uscita di un latch, attraverso una rete combinatoria, torna all'ingresso dello stesso latch, finché l'abilitazione è attiva i segnali girano nell'anello senza fermarsi, come nel latch JK. Per questo nelle reti sincrone si usano elementi che campionano l'ingresso in un solo istante: i flip-flop del paragrafo 3.3.

Se l'ingresso di un latch cambia proprio mentre l'abilitazione si chiude, l'uscita può restare per un tempo non prevedibile a un livello intermedio prima di decidere fra 0 e 1. È la **metastabilità**, la versione reale dell'oscillazione del latch SR, e si evita rispettando i tempi del paragrafo 3.4.

3.3 Flip-flop sul fronte

Un **flip-flop** (FF) cambia stato solo in corrispondenza di un **fronte** del clock CK: il fronte di salita, da 0 a 1, per i FF **PET** (*positive edge triggered*), il fronte di discesa per i FF **NET** (*negative edge triggered*). Fra un fronte e l'altro l'uscita resta ferma qualunque cosa facciano gli ingressi. Nei simboli il fronte si indica con un triangolino sull'ingresso del clock, e un pallino davanti al triangolino indica un FF NET. In questo corso si usano i FF PET.

Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-04)

Come si ottiene il fronte. Un modo è lo schema **master-slave**: due latch D in serie, il primo (master) abilitato quando $CK = 0$ e il secondo (slave) quando $CK = 1$. Mentre il clock è basso il master segue l'ingresso e lo slave è chiuso; quando il clock sale il master si chiude sul valore che aveva in quel momento e lo slave lo copia in uscita. L'uscita cambia quindi solo sul fronte di salita, e non c'è mai un percorso trasparente dall'ingresso all'uscita.

I tre FF che servono sono il D, il JK e il T, tutti con la stessa idea: la tabella dice quale sarà lo stato dopo il prossimo fronte.

FF	Ingressi	Q^+	Equazione caratteristica
D	$D = 0$ / $D = 1$	0 / 1	$Q^+ = D$
JK	$JK = 00, 01, 10, 11$	Q, 0, 1, $\bar{Q}$	$Q^+ = J\bar{Q} + \bar{K}Q$
T	$T = 0$ / $T = 1$	Q / $\bar{Q}$	$Q^+ = T \oplus Q$

Nel FF JK la combinazione $J = K = 1$ non fa più oscillare: lo stato si inverte una volta per fronte. Il FF **T** (*toggle*) si ottiene da un JK con $J = K = T$, oppure da un D con $D = T \oplus Q$.

I FF hanno spesso due ingressi **asincroni**, attivi bassi, che agiscono subito senza aspettare il clock: $\overline{PR}$ (*preset*) forza lo stato a 1 quando vale 0, $\overline{CL}$ (*clear*) lo forza a 0. Si usano per l'inizializzazione: il segnale $\overline{RES}$ di una rete va sui $\overline{CL}$ di tutti i FF, e quando non servono si collegano a 1.

Per il progetto serve la tabella inversa di quella caratteristica: data la transizione $Q \to Q^+$ che si vuole ottenere, quali ingressi bisogna dare. È la **tabella di eccitazione**.

$Q \to Q^+$	D	J	K	T
0 → 0	0	0	–	0
0 → 1	1	1	–	1
1 → 0	0	–	1	1
1 → 1	1	–	0	0

Per il JK ci sono sempre indifferenze: per passare da 0 a 1 si può dare il set ($JK = 10$) o la commutazione ($JK = 11$), quindi K non conta. Le indifferenze sono il motivo per cui le reti che comandano i JK vengono spesso più semplici di quelle che comandano i D.

La tabella che dagli ingressi dà lo stato successivo è la tabella **caratteristica**; quella che dalla transizione dà gli ingressi è la tabella di **eccitazione**. Negli esercizi si usano entrambe, e non vanno confuse.

3.4 Temporizzazioni

Un FF reale funziona correttamente solo se gli ingressi rispettano alcuni tempi rispetto al fronte del clock (figura 3.4).

Parametri temporali di un flip-flop

- **Tempo di setup** t_s (minimo): l'ingresso deve essere stabile almeno da t_s prima del fronte.
- **Tempo di hold** t_h (minimo): l'ingresso deve restare stabile almeno per t_h dopo il fronte.
- **Tempo di propagazione** t_p (massimo): l'uscita cambia al più t_p dopo il fronte; si distingue t_{PLH} per la salita dell'uscita e t_{PHL} per la discesa.
- **Durata minima dell'impulso** t_w: gli impulsi del clock e dei comandi asincroni devono durare almeno t_w.

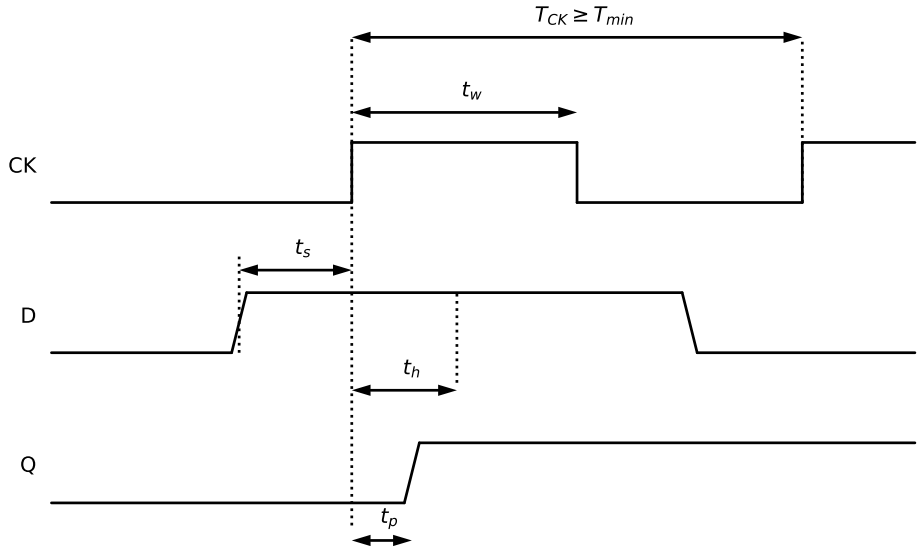


Figura 3.4

Se uno di questi tempi non è rispettato il FF può prendere il valore sbagliato o diventare metastabile. Da questi parametri dipende la velocità massima di una rete sincrona.

Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-05)

Periodo minimo di una rete sincrona. In una rete in cui le uscite dei FF passano per una rete combinatoria e tornano agli ingressi dei FF, dopo un fronte le uscite cambiano entro t_p, la rete combinatoria le elabora entro il suo ritardo massimo t_{comb}, e il risultato deve arrivare agli ingressi almeno t_s prima del fronte successivo. Il periodo del clock deve quindi rispettare

$$T_{CK} \geq T_{min} = t_p + t_{comb} + t_s, \qquad f_{max} = \frac{1}{T_{min}}.$$

Con $t_p = 10$ ns, $t_{comb} = 25$ ns e $t_s = 5$ ns si ha $T_{min} = 40$ ns e $f_{max} = 25$ MHz. C'è anche un vincolo dall'altra parte: il nuovo valore non deve arrivare troppo presto, prima che sia passato il tempo di hold del fronte appena avvenuto. Serve che il percorso più veloce rispetti $t_{p,min} + t_{comb,min} \geq t_h$, condizione che con i FF commerciali è quasi sempre soddisfatta.

> T_{min} è un periodo, e si misura in secondi; la frequenza massima del clock è il suo inverso. Un clock a 1 MHz ha periodo 1 μs, non 1 ms.

3.5 Registri

Un **registro** a n bit è un gruppo di n FF D con lo stesso clock, che memorizza una parola di n bit. A seconda di come entrano ed escono i dati si hanno quattro tipi.

- **PIPO** (*parallel input, parallel output*): ogni FF ha il suo ingresso P_i e la sua uscita Q_i; a ogni fronte il registro carica tutti i bit insieme. Si disegna come un blocco con un bus di ingresso e uno di uscita a n fili.
- **SISO** (*serial input, serial output*): i FF sono in fila, l'uscita di ciascuno è l'ingresso del successivo, il dato entra da S_{in} un bit per fronte ed esce da S_{out} dopo n fronti. È il **registro a scorrimento** (*shift register*) (figura 3.5).
- **SIPO** (*serial input, parallel output*): è un SISO di cui si prendono tutte le uscite; converte un dato seriale in parallelo.
- **PISO** (*parallel input, serial output*): davanti a ogni FF un selettore comandato da LD sceglie fra l'ingresso parallelo P_i (caricamento, $LD = 1$) e l'uscita del FF precedente (scorrimento, $LD = 0$); converte un dato parallelo in seriale.

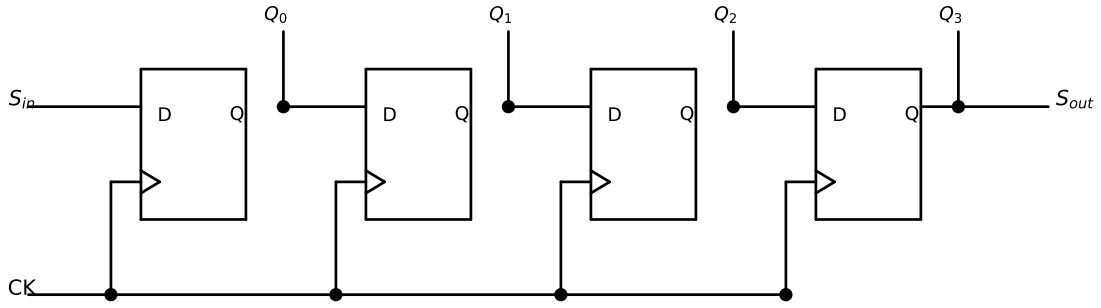


Figura 3.5

Il **registro universale** mette davanti a ogni FF un MUX 4→1 che sceglie che cosa caricare al prossimo fronte. Numerando i bit da destra (Q_0 è l'LSB e Q_{n-1} il più a sinistra), il bit i riceve:

S_0S_1	**Ingresso del FF i**	**Operazione**
00	Q_i	memorizzazione
01	Q_{i+1}	scorrimento a destra
10	Q_{i-1}	scorrimento a sinistra
11	P_i	caricamento parallelo

Ai due estremi gli ingressi mancanti (Q_n per il bit più a sinistra, Q_{-1} per il bit 0) sono ingressi seriali esterni.

3.6 Contatori

Un **contatore** è un registro che a ogni fronte passa al numero successivo. Nella sequenza binaria il bit Q_0 cambia a ogni passo, e il bit Q_i cambia quando tutti i bit alla sua destra valgono 1:

N	$Q_3Q_2Q_1Q_0$		N	$Q_3Q_2Q_1Q_0$
0	0000		5	0101
1	0001		6	0110
2	0010		7	0111
3	0011		8	1000
4	0100		9	1001

Con i FF T la regola si traduce direttamente: $T_0 = 1$, $T_1 = Q_0$, $T_2 = Q_1 Q_0$, $T_3 = Q_2 Q_1 Q_0$. È il **contatore sincrono** a 4 bit, che conta da 0 a 15 e poi riparte (contatore modulo 16, figura 3.6). Tutti i FF ricevono lo stesso clock, quindi le uscite cambiano insieme.

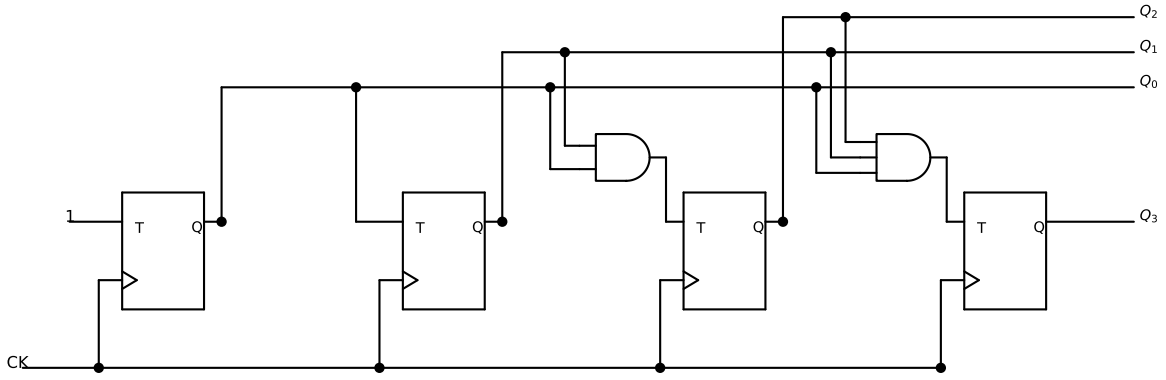


Figura 3.6

Con i FF D lo stesso contatore si progetta dalle equazioni. Per 2 bit: Q_0 si inverte sempre, $D_0 = \bar{Q}_0$ (il FF è in configurazione *toggle*); Q_1 si inverte quando $Q_0 = 1$, $D_1 = Q_1 \oplus Q_0$. Il contatore modulo 4 conta $0, 1, 2, 3, 0, \ldots$ e ha un'uscita **TC** (*terminal count*) che vale 1 nell'ultimo stato, $TC = Q_1 Q_0$ (figura 3.7).

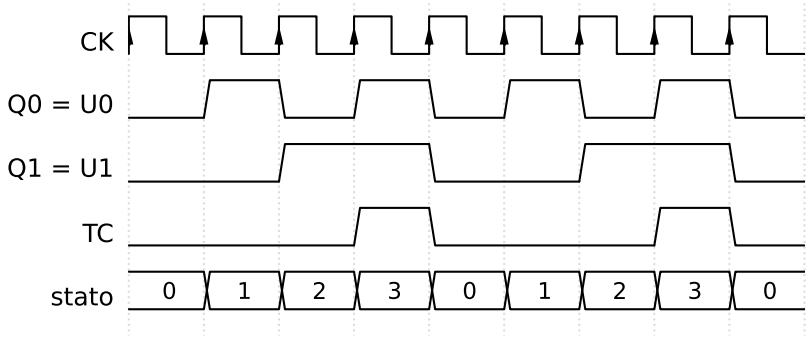


Figura 3.7

Un **contatore universale** si ottiene con un registro e un sommatore: a ogni fronte il registro carica $Q + N$. Con $N = 00\ldots01$ conta in avanti; con $N = 11\ldots11$, che in complemento a 2 vale -1, conta all'indietro. Il terminal count del conteggio in avanti è lo stato con tutti i bit a 1, $TC = Q_{k-1} Q_{k-2} \cdots Q_1 Q_0$; nel conteggio all'indietro l'ultimo stato è quello con tutti i bit a 0.

> **Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-05)**
>
> **Contatore modulo N e cascata.** Per contare da 0 a $N - 1$ con N non potenza di 2 si riconosce lo stato $N - 1$ e si forza lo stato successivo a 0. Per il contatore decimale (0…9), fra gli stati usati solo il 9 (1001) ha $Q_3 = 1$ e $Q_0 = 1$, quindi $TC = Q_3 Q_0$, e lo stato successivo è $Q^+ = \overline{TC} \cdot (Q + 1)$: un reset sincrono comandato da TC.
>
> Un contatore con **abilitazione** EN conta solo quando $EN = 1$ e dà TC solo se è abilitato. Collegando il TC di un contatore all'EN del successivo si ottiene un contatore con più cifre: il secondo avanza di uno ogni volta che il primo ricomincia, come le cifre di un contachilometri.

3.7 Analisi di reti sincrone

Analizzare una rete sincrona vuol dire ricavarne il comportamento dal circuito. Si scrivono le equazioni degli ingressi dei FF in funzione degli ingressi della rete e delle uscite dei FF, si calcola lo stato dopo ogni fronte, e si disegna il diagramma temporale. Conviene ricordare due regole: i valori si leggono **appena prima** del fronte, e le uscite dei FF cambiano **appena dopo**.

Due flip-flop in cascata

L'ingresso IN va a un primo FF, la cui uscita U_0 va all'ingresso D di un secondo FF con uscita U_1; i due FF hanno lo stesso clock e lo stesso $\overline{RES}$. Finché $\overline{RES} = 0$ le uscite sono a 0. Poi, a ogni fronte, U_0 prende il valore che IN aveva appena prima del fronte, e U_1 quello che U_0 aveva appena prima: U_1 è U_0 ritardato di un periodo di clock. Un impulso di IN che comincia e finisce fra due fronti non viene visto affatto.

Negli appunti (L08 e L07) il primo FF è un JK con IN su J, ma non si legge dove sia collegato K. Se $K = \overline{IN}$, il JK si comporta come un FF D e il diagramma è quello descritto sopra. Va confermato sul circuito.

Un registro a scorrimento con retroazione

Quattro FF D sono in fila con uscite D, C, B, A (il primo è D). L'ingresso del primo FF è $IN \oplus B \oplus A$, l'uscita di ogni FF è l'ingresso del successivo; il reset porta tutto a 0000 (figura 3.8).

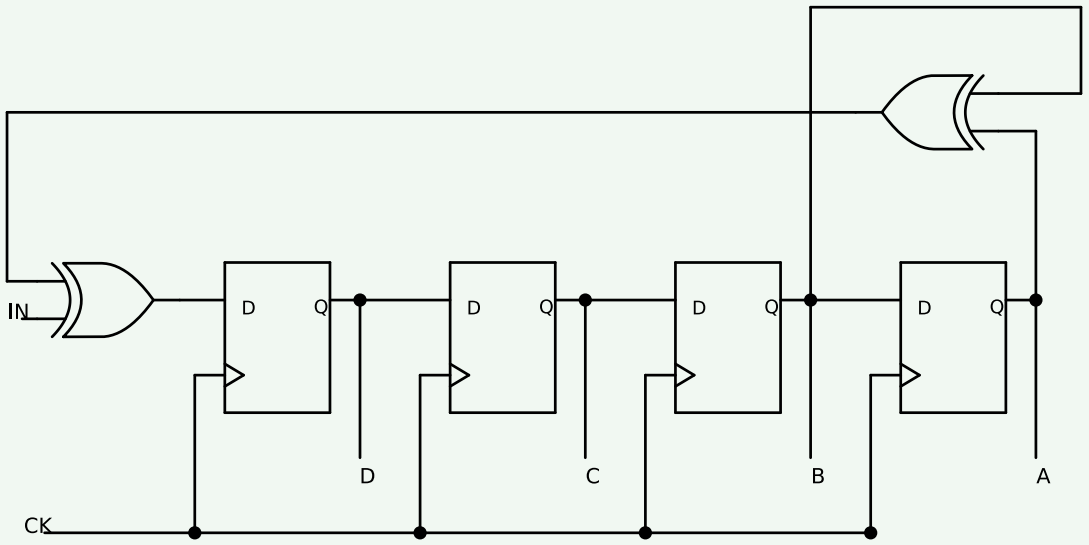


Figura 3.8

Lo stato successivo di $DCBA$ è $(IN \oplus B \oplus A,\ D,\ C,\ B)$. Con IN fisso a 1 dopo il reset si ottiene la sequenza

$0000 \to 1000 \to 1100 \to 1110 \to 0111 \to 1011 \to 1101 \to 0110 \to 0011 \to 1001 \to 0100 \to 1010 \to 0101 \to 0010 \to 0001 \to 0000$

che si ripete ogni 15 fronti (lo stato 1111 non compare). Con $IN = 1$ per un solo fronte e poi sempre 0, la rete parte da 1000 e attraversa tutti i 15 stati diversi da 0000:

$1000 \to 0100 \to 0010 \to 1001 \to 1100 \to 0110 \to 1011 \to 0101 \to 1010 \to 1101 \to 1110 \to 1111 \to 0111 \to 0011 \to 0001 \to 1000$

È un **registro a scorrimento con retroazione lineare** (LFSR): con la sola XOR sulla retroazione genera una sequenza lunga, che sembra casuale, e si usa per esempio per produrre bit di prova. Se IN resta a 0 dal reset in poi, la rete resta in 0000 per sempre: serve almeno un 1 per farla partire.

Esempi ed esercizi svolti

Esercizio 1. Completare la tabella del latch SR a NAND. Gli ingressi sono $\bar{S}$, $\bar{R}$, attivi bassi: $\bar{S}\bar{R} = 11$ memorizza, 01 (set attivo) dà $Q = 1$, 10 (reset attivo) dà $Q = 0$, 00 non è ammesso perché porta entrambe le uscite a 1.

Esercizio 2. Ottenere un FF T da un FF D. Dalla tabella di eccitazione del D, $D = Q^+$; dal T, $Q^+ = T \oplus Q$. Quindi $D = T \oplus Q$: una XOR fra T e l'uscita del FF davanti all'ingresso D.

Esercizio 3. Quale frequenza massima ha un contatore sincrono a 4 bit con FF T, se $t_p = 8$ ns, $t_s = 4$ ns e ogni AND ha un ritardo di 5 ns? Il percorso più lungo va da Q_0, Q_1, Q_2 attraverso la AND che dà T_3: se la AND è una sola porta a tre ingressi, $t_{comb} = 5$ ns e $T_{min} = 8 + 5 + 4 = 17$ ns, cioè $f_{max} \approx 58{,}8$ MHz. Se invece T_3 si ottiene in cascata, come $T_3 = T_2 \cdot Q_2$ con due AND in serie, $t_{comb} = 10$ ns e $T_{min} = 22$ ns ($f_{max} \approx 45{,}5$ MHz).

Esercizio 4. Un registro universale a 4 bit contiene $Q_3Q_2Q_1Q_0 = 1011$, con ingresso seriale a destra 0 e a sinistra 0. Dopo un fronte con $S_0S_1 = 01$ (scorrimento a destra, ogni bit prende il valore del vicino di sinistra) contiene 0101; dopo un fronte con $S_0S_1 = 10$ (a sinistra) partendo da 1011 contiene 0110.

Riepilogo

Una retroazione trasforma una rete combinatoria in un elemento di memoria: il latch SR, con due NOR o due NAND incrociate, conserva lo stato e risponde ai comandi di set e reset, con una combinazione vietata. Con l'abilitazione si ottengono il latch D, senza combinazioni vietate, e il latch JK, che però oscilla con $J = K = 1$ perché è trasparente. I flip-flop cambiano solo sul fronte del clock e risolvono il problema; i tipi D, JK e T hanno equazioni caratteristiche $Q^+ = D$, $Q^+ = J\bar{Q} + \bar{K}Q$, $Q^+ = T \oplus Q$, e per il progetto si usa la tabella di eccitazione. I tempi di setup, hold, propagazione e durata minima vanno rispettati, e il periodo minimo di una rete sincrona è $t_p + t_{comb} + t_s$. I registri memorizzano parole di bit e possono caricare in parallelo o scorrere, e il registro universale sceglie l'operazione con un MUX per bit. I contatori sincroni si fanno con FF T (T_i uguale al prodotto dei bit più bassi) o con FF D, segnalano l'ultimo stato con il terminal count e si collegano in cascata con l'abilitazione. Per analizzare una rete sincrona si scrivono le equazioni dei FF e si calcola lo stato fronte per fronte.

4 · Macchine a stati finiti e progetto con ASM

In questo modulo. Si usano le sintesi con le mappe del modulo 1 (*Algebra booleana e sintesi delle reti combinatorie*), i blocchi standard del modulo 2 (*Reti combinatorie standard, ritardi e aritmetica*) e i flip-flop, i registri e i contatori del modulo 3 (*Flip-flop, registri e contatori*). Si vede che ogni rete sequenziale sincrona è una **macchina a stati finiti**, nelle due varianti di Moore e di Mealy, e si impara a descriverla con i diagrammi **ASM**. Poi si segue la procedura completa di progetto, dalla specifica al circuito con FF D o JK, e quella inversa di analisi. Seguono le strutture che ricorrono negli esercizi, la gestione degli stati non usati, alcuni progetti completi, l'architettura controllore-datapath e la verifica con il simulatore. Alla fine del modulo si sa progettare a livello funzionale un semplice sistema digitale basato su macchine a stati finiti, che è il risultato atteso dal corso.

4.1 La macchina a stati finiti

Una **macchina a stati finiti** (MSF, in inglese FSM) sincrona è composta da tre parti (figura 4.1):

- il **registro di stato**, fatto di k FF con lo stesso clock e lo stesso reset, che memorizza lo stato presente $X = X_{k-1} \dots X_0$;
- la **rete di calcolo dello stato successivo**, combinatoria, che dagli ingressi I e dallo stato presente calcola lo stato successivo X^+, che il registro carica al prossimo fronte;
- la **rete di calcolo delle uscite**, combinatoria, che produce le uscite U.

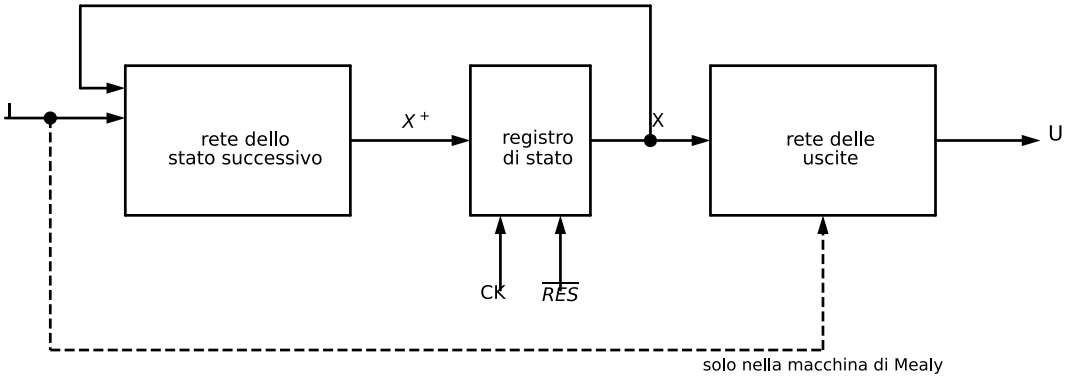


Figura 4.1

Con k FF gli stati possibili sono al più 2^k, da cui il nome. Nella **macchina di Moore** le uscite dipendono solo dallo stato presente, $U = f(X)$; nella **macchina di Mealy** dipendono anche dagli ingressi, $U = f(X, I)$, e la rete delle uscite riceve anche gli ingressi. Si suppone sempre che gli ingressi siano **sincronizzati** con il clock, cioè che cambino lontano dai fronti; un ingresso che arriva dall'esterno, come un pulsante, si sincronizza prima con un FF (paragrafo 4.5).

Il funzionamento si legge sul diagramma temporale: a ogni fronte il registro carica lo stato successivo, nel periodo seguente la rete combinatoria calcola il nuovo stato successivo dagli ingressi e dallo stato, e così via. Lo stato è una configurazione di bit, ma per ragionare conviene dargli un nome (a, b, c, …) e scrivere sul diagramma la sequenza degli stati invece dei singoli bit.

4.2 Il linguaggio ASM

Il diagramma **ASM** (*algorithmic state machine*) è un linguaggio grafico per descrivere il funzionamento di una MSF, simile a un diagramma di flusso ma con un significato temporale preciso. Gli elementi sono tre (figura 4.2).

Elementi del diagramma ASM

- Il **blocco di stato** è un rettangolo, uno per ogni stato, con il nome dello stato a lato e il suo codice sopra. Dentro si scrivono le uscite che valgono 1 in quello stato (le uscite di Moore); le uscite non scritte valgono 0.
- Il **blocco decisionale** è un rombo con una condizione sugli ingressi e due uscite, 1 e 0. Si attraversa nello stesso periodo di clock dello stato da cui parte.
- Il **blocco di uscita condizionata** è un ovale dopo un rombo, con le uscite che valgono 1 quando la macchina è nello stato e la condizione è vera (le uscite di Mealy).

Lo stato di reset si indica con un segno sulla freccia che entra nel suo blocco. Ogni periodo di clock corrisponde al percorso da un blocco di stato al successivo: il blocco di stato con i rombi e gli ovali che lo seguono si chiama **blocco ASM**.

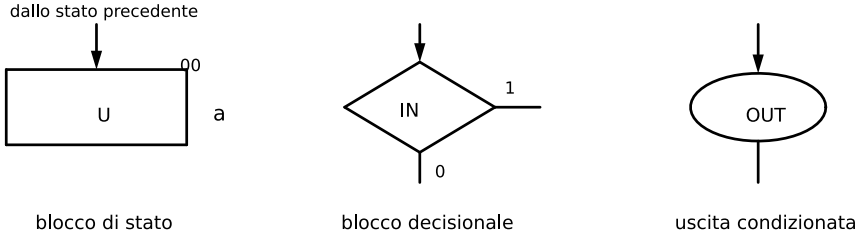


Figura 4.2

Contatore modulo 4

Il contatore ha uscite U_1, U_0 e TC, e conta $0, 1, 2, 3, 0, \ldots$ con $TC = 1$ nell'ultimo stato. L'ASM ha quattro blocchi di stato in fila, senza rombi, perché la macchina non ha ingressi oltre al clock e al reset (figura 4.3): nello stato 0 non c'è nessuna uscita, nello stato 1 c'è U_0, nello stato 2 U_1, nello stato 3 TC, U_1, U_0.

Stato	TC	U_1	U_0
0	0	0	0
1	0	0	1
2	0	1	0
3	1	1	1

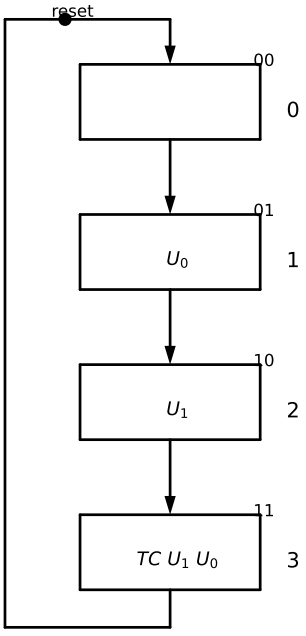


Figura 4.3

Contatore avanti e indietro

Con un ingresso U/D il contatore conta in avanti, $0, 1, 2, 3, 0, \ldots$, se $U/D = 1$, e all'indietro, $3, 2, 1, 0, 3, \ldots$, se $U/D = 0$. L'ASM ha gli stessi quattro blocchi di stato, ma dopo ciascuno c'è un rombo su U/D: con 1 si va allo stato successivo, con 0 al precedente (figura 4.4).

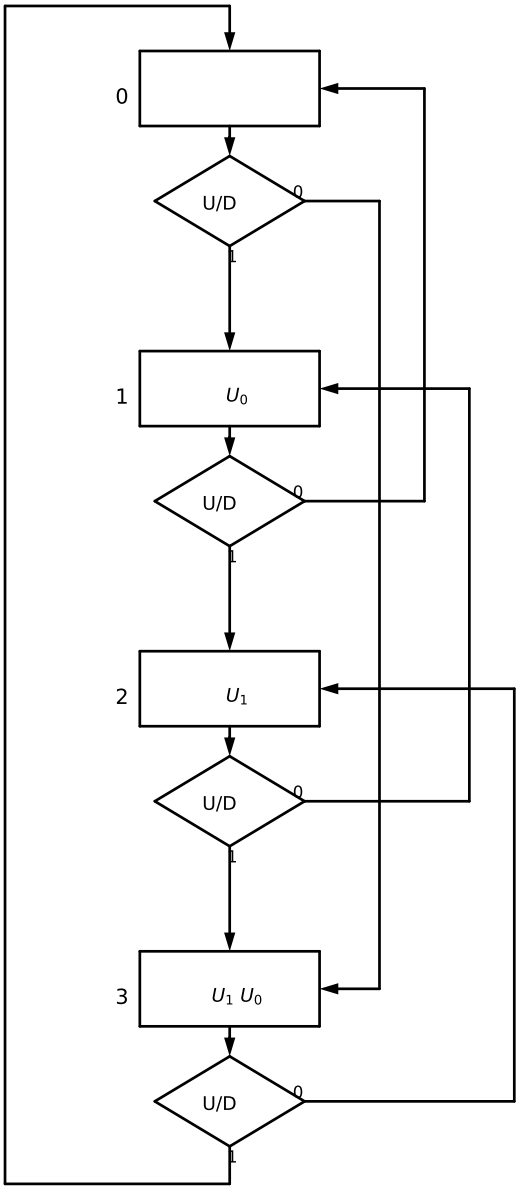


Figura 4.4

4.3 La procedura di progetto

Il progetto di una MSF segue sempre gli stessi passi.

0. **Specifica**: si capisce che cosa deve fare la macchina, quali sono gli ingressi e le uscite, e lo si scrive in modo preciso, di solito con un diagramma temporale di esempio.
1. **Descrizione formale**: si disegna l'ASM. Esistono anche linguaggi testuali di descrizione dell'hardware, come VHDL e Verilog, che servono allo stesso scopo.
2. **Registro di stato**: si sceglie il numero di FF, almeno $\lceil \log_2(\text{numero di stati}) \rceil$, e si assegna un codice a ogni stato.
3. **Reti combinatorie**: si scrivono la tabella delle uscite e la tabella degli stati (stato presente e ingressi → stato successivo), si ricavano gli ingressi dei FF (con i D coincidono con lo stato successivo, con i JK si usa la tabella di eccitazione) e si sintetizzano con le mappe.

Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-06)

Scelta della codifica. Codifiche diverse danno reti diverse, e non c'è una regola che garantisca la migliore. Alcuni criteri pratici: dare codici adiacenti (che differiscono per un bit) a stati collegati da una transizione; dare allo stato di reset il codice $00\ldots0$, così il reset si fa con i $\overline{CL}$; scegliere i codici in modo che le uscite coincidano con qualche bit dello stato, così la rete delle uscite sparisce. All'estremo opposto c'è la codifica **one-hot**, con un FF per ogni stato e un solo FF a 1 alla volta: più FF, ma reti dello stato successivo molto semplici.

Sequenza di luci

Specifica. Tre luci A, B, C (1 accesa, 0 spenta) si accendono in sequenza: nessuna, A, A e B, tutte e tre, B e C, solo C, poi si ricomincia. La macchina non ha ingressi.

ASM. Sei stati in fila, senza rombi: a (nessuna uscita), b (A), c (A, B), d (A, B, C), e (B, C), f (C).

Registro di stato. Sei stati richiedono 3 FF, con variabili X, Y, Z. Si usano i codici binari in ordine: a = 000, b = 001, c = 010, d = 011, e = 100, f = 101. I codici 110 e 111 non sono usati e nelle mappe sono indifferenze.

Uscite. Mappe con righe X e colonne YZ:

$X\backslash YZ$	**00**	**01**	**11**	**10**
A: 0	0	1	1	1
A: 1	0	0	–	–
B: 0	0	0	1	1
B: 1	1	0	–	–
C: 0	0	0	1	0
C: 1	1	1	–	–

Da cui $A = Y + \bar{X}Z$, $B = Y + X\bar{Z}$, $C = X + YZ$.

Stato successivo.

Stato	XYZ	$X^+Y^+Z^+$
a	000	001
b	001	010
c	010	011
d	011	100
e	100	101
f	101	000

Con i FF D gli ingressi sono le colonne dello stato successivo, e con le mappe (indifferenze in 110 e 111): $D_X = X\bar{Z} + YZ$, $D_Y = Y\bar{Z} + \bar{X}\bar{Y}Z$, $D_Z = \bar{Z}$.

Con i FF JK si applica la tabella di eccitazione bit per bit. Per X: nelle righe con $X = 0$ serve $J_X = 1$ solo nella transizione $011 \to 100$, e nelle righe con $X = 1$ serve $K_X = 1$ solo in $101 \to 000$; le altre celle sono indifferenze. Si ottengono $J_X = YZ$, $K_X = Z$, $J_Y = \bar{X}Z$, $K_Y = Z$, $J_Z = K_Z = 1$: il FF di Z commuta a ogni fronte. Con i JK la rete dello stato successivo è più semplice, come anticipato nel paragrafo 3.3.

4.4 Analisi di una MSF

L'analisi fa il percorso inverso: dal circuito alla descrizione del comportamento.

Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-06)

Procedura di analisi. (1) Si individuano i FF che formano il registro di stato, la rete dello stato successivo e la rete delle uscite. (2) Si scrivono le equazioni degli ingressi dei FF e delle uscite in funzione dello stato e degli ingressi. (3) Si ricava la tabella degli stati: per ogni stato presente e ogni combinazione degli ingressi, lo stato successivo (con i JK attraverso l'equazione caratteristica). (4) Si dà un nome a ogni stato e si disegna l'ASM. (5) Si controlla il risultato con un diagramma temporale.

Dal circuito all'ASM

Un circuito ha due FF D con uscite X_1, X_0, un ingresso R e le reti

$$D_0 = \bar{R}\,\bar{X}_0, \qquad D_1 = \bar{R}\,(X_0 \oplus X_1), \qquad U_0 = X_0, \quad U_1 = X_1, \quad TC = X_1 X_0.$$

Con $R = 0$ lo stato successivo è $X_1^+ X_0^+ = (X_1 \oplus X_0,\ \bar{X}_0)$, cioè $00 \to 01 \to 10 \to 11 \to 00$: è il contatore modulo 4. Con $R = 1$ lo stato successivo è sempre 00. Gli stati si chiamano a (00), b (01), c (10), d (11); nell'ASM ogni blocco di stato è seguito da un rombo su R, che con 0 porta allo stato successivo e con 1 riporta in a. È un contatore modulo 4 con **reset sincrono**: a differenza del $\overline{RES}$ sui $\overline{CL}$, il comando R agisce solo al fronte.

4.5 Strutture base

Alcune macchine piccole ricorrono in quasi tutti gli esercizi, da sole o come parti di sistemi più grandi.

Sincronizzatore. Specifica: l'uscita OUT vale 1 per il tempo in cui l'ingresso IN vale 1, ma cambia solo sui fronti. Bastano due stati: a, con $OUT = 0$, in cui si resta finché $IN = 0$ e da cui si passa in b quando $IN = 1$; b, con OUT, in cui si resta finché $IN = 1$. È un solo FF D con $D = IN$. Un impulso di IN che comincia e finisce fra due fronti non viene visto: gli ingressi devono variare con una frequenza molto più bassa di quella del clock.

Rilevatore di fronti. Specifica: OUT vale 1 per un periodo di clock dopo ogni fronte di IN, sia di salita sia di discesa (figura 4.5). In una macchina di Moore servono quattro stati: a (IN era 0), b (IN è appena salito: OUT), c (IN è 1 da più di un periodo), d (IN è appena sceso: OUT). Da a si va in b quando $IN = 1$; da b e da c si va in c se $IN = 1$ e in d se $IN = 0$; da d si va in b se $IN = 1$ e in a se $IN = 0$. In una macchina di Mealy bastano due stati, che ricordano il valore precedente di IN: lo stato è un FF D con $D = IN$, e l'uscita condizionata è $OUT = IN \oplus X$, che vale 1 finché il nuovo valore di IN è diverso da quello memorizzato.

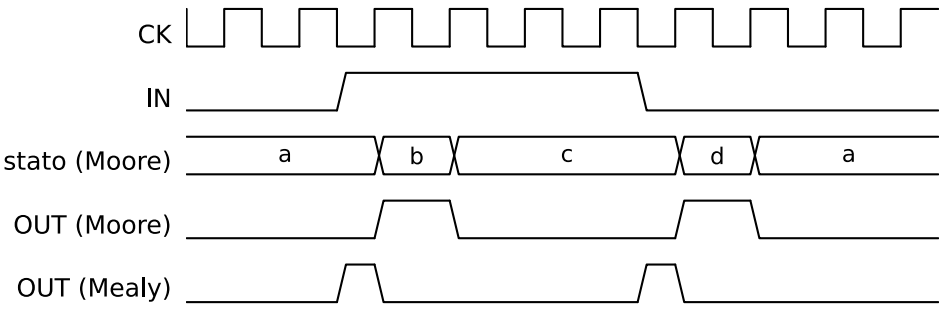


Figura 4.5

Negli appunti (L09) il rilevatore di fronti è indicato come "macchina a 2 stati", ma il diagramma temporale usa quattro stati (a, b, c, d). Le due cose tornano se i 2 stati sono quelli della versione di Mealy: va confermato quale versione è stata fatta a lezione.

Rilevatore di impulso lungo. Specifica: $OUT = 1$ per un periodo se IN resta a 1 per almeno due periodi. Stati: a (attesa, si resta finché $IN = 0$), b (IN è 1 da un fronte: se torna a 0 si torna in a), c (OUT: IN è 1 da due fronti), d (attesa che IN torni a 0, senza uscita). Da c si va in d se $IN = 1$ e in a se $IN = 0$.

Generatore di sequenze. Specifica: alla transizione di IN da 0 a 1 la macchina genera sulle uscite A, B la sequenza $A = 1$, poi $B = 1$, poi $A = B = 1$, e torna in attesa. Stati: a (attesa, rombo su IN), b (A), c (B), d (A, B),

e (attesa che IN torni a 0, rombo su IN). Lo stato e serve a reagire alla *transizione* e non al livello: senza, se IN restasse a 1 la sequenza ripartirebbe subito.

Rilevatore di due ingressi. Specifica: la macchina genera una sequenza quando entrambi gli ingressi A e B valgono 1. Nell'ASM si può scrivere un rombo per A seguito da un rombo per B, oppure un solo rombo con la condizione $A \cdot B$: le due forme sono equivalenti.

4.6 Moore e Mealy a confronto

In una macchina di Moore le uscite cambiano solo dopo un fronte, perché dipendono solo dallo stato: sono sincrone e prive delle alee degli ingressi, ma arrivano un periodo dopo la condizione che le provoca. In una macchina di Mealy le uscite condizionate reagiscono subito al cambiamento degli ingressi, nello stesso periodo (si dice che l'uscita è **anticipata**), e spesso servono meno stati; in cambio le uscite seguono gli ingressi in modo asincrono, con le loro variazioni e le loro alee (figura 4.6).

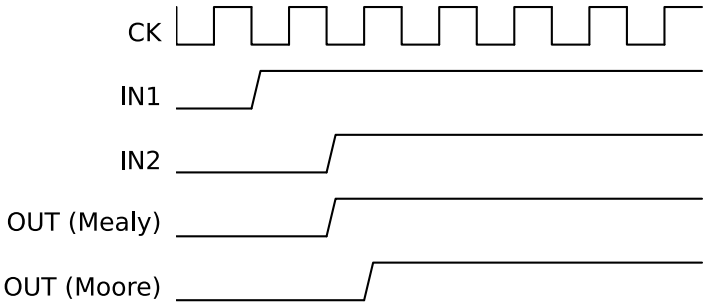


Figura 4.6

Lo stesso problema in tre macchine

Specifica. Ingressi ENB, SEL; uscite GO, HLT. La macchina attende $ENB = 1$; poi attiva GO se $SEL = 1$ e HLT se $SEL = 0$. Le uscite restano attive finché $ENB = 1$; quando ENB torna a 0 la macchina torna in attesa.

M1, Moore, tre stati. a (attesa) con un rombo su ENB e uno su SEL; b (HLT) e c (GO), ciascuno con un rombo su ENB che lo tiene fermo finché $ENB = 1$ e lo riporta in a quando torna a 0. Con a = 00, b = 01, c = 10: $HLT = Y$, $GO = X$ e, con l'indifferenza in 11,

$$D_X = ENB\,(X + SEL\,\bar{Y}), \qquad D_Y = ENB\,(Y + \overline{SEL}\,\bar{X}).$$

M2 e M3, Mealy, due stati. a (attesa) e b (attivo), con un solo FF: $D_X = ENB$. Nello stato b le uscite sono condizionate da SEL: $GO = SEL \cdot X$, $HLT = \overline{SEL} \cdot X$.

Le macchine non sono equivalenti: in M1 il valore di SEL si legge una volta, quando ENB sale, e dopo non conta più; in M2 e M3 le uscite seguono SEL per tutto il tempo in cui $ENB = 1$, quindi se SEL cambia nel frattempo l'uscita attiva cambia. M2 e M3 sono due disegni dello stesso comportamento: $M1 \neq M2 \sim M3$. Quale sia giusta dipende da come si legge la specifica, ed è il tipo di ambiguità da chiarire al passo 0.

Impulso sul fronte e segnalazione di attesa

Specifica. Ingresso IN; uscite OUT e WT. OUT è un impulso di un periodo alla transizione di IN da 0 a 1; WT si attiva a 1 se IN resta a 1 per più di un periodo, e resta attivo (la macchina si blocca) fino al reset.

ASM e codifica. a = 00 (attesa: con $IN = 1$ va in b), b = 01 (OUT: con $IN = 0$ torna in a, con $IN = 1$ va in c), c = 10 (WT, resta in c). Il codice 11 non è usato.

Stato	IN	XY	X^+Y^+
a	0	00	00
a	1	00	01
b	0	01	00
b	1	01	10
c	–	10	10

Uscite: $OUT = Y$, $WT = X$. Con i FF D: $D_X = X + IN \cdot Y$, $D_Y = IN\,\bar{X}\,\bar{Y}$. Con i FF JK, dalla tabella di eccitazione: $J_X = IN \cdot Y$, $K_X = 0$, $J_Y = IN \cdot \bar{X}$, $K_Y = 1$.

Nella mappa di J_X le colonne sono quelle di Y: l'1 condizionato ($J_X = IN$) è nella cella $XY = 01$, quindi $J_X = IN \cdot Y$. Con $J_X = IN \cdot \bar{X}$ la macchina passerebbe da a direttamente in c al primo fronte con $IN = 1$.

4.7 Stati non usati ed errori

Con k FF e meno di 2^k stati restano dei codici non usati. Nella sintesi conviene trattarli come indifferenze, ma così lo stato successivo di un codice non usato è quello che la minimizzazione ha scelto per caso. In condizioni normali la macchina non ci arriva mai; ci può arrivare però per un disturbo, per esempio un **bit flip** di un FF dovuto a un impulso elettrico o a una radiazione, o all'accensione se il reset manca. Il pericolo è che i codici non usati formino un ciclo da cui la macchina non esce più.

Per la sequenza di luci del paragrafo 4.3 il controllo si fa sulle equazioni: da 110 si va in 111, e da 111 in 100, che è lo stato e. La macchina rientra da sola nella sequenza dopo due fronti.

Quando non basta, il progetto si rende più sicuro definendo anche le transizioni dei codici non usati. Per una macchina a tre stati a (000), b (001), c (010) con 3 FF, i cinque codici liberi possono diventare stati di errore ERR, ciascuno con un anello su se stesso e un'uscita di allarme, oppure portare direttamente allo stato di reset. Si rinuncia a qualche indifferenza, e la rete viene un po' più grande.

4.8 Progetti completi

Controllo di una lavatrice

Specifica. Ingressi $START$ (inizio del lavaggio sulla transizione da 0 a 1) e L (0 lavaggio normale, 1 lavaggio lungo); uscite LAV (lavaggio), RIS (risciacquo), CEN (centrifuga). Ogni fase dura un periodo di clock, e il lavaggio lungo ha due periodi di LAV invece di uno.

ASM. Una prima versione ha due rami separati dopo il rombo su L: LAV, RIS, CEN per il normale e LAV, LAV, RIS, CEN per il lungo. Si ottimizza osservando che i rami finiscono allo stesso modo: il ramo lungo fa un periodo di LAV in più e poi si unisce al normale. Stati: wait (000, rombo su $START$ e poi su L), LL (001, LAV, solo per il lavaggio lungo), L1 (010, LAV), L2 (011, RIS), L3 (100, CEN), stop (101). Da wait si va in LL se $START \cdot L$, in L1 se $START \cdot \bar{L}$; poi LL → L1 → L2 → L3 → stop.

Uscite, con le indifferenze in 110 e 111: $LAV = Y\bar{Z} + \bar{X}\bar{Y}Z$, $RIS = YZ$, $CEN = X\bar{Z}$.

Sintesi con i FF JK. Dalla tabella di eccitazione, bit per bit:

$$J_X = YZ, \quad K_X = 0; \qquad J_Y = \bar{X}Z + \bar{X}\,\bar{Z} \cdot START\,\bar{L}, \quad K_Y = Z; \qquad J_Z = X + Y + START \cdot L, \quad K_Z = \bar{X}.$$

J_Y si può scrivere anche $\bar{X}(Z + START\,\bar{L})$. Lo stato stop resta fermo: $K_Z = \bar{X}$ vale 0 per $X = 1$, e J_X, J_Y valgono 0.

Nella mappa di J_Z le celle con $X = 1$ o $Y = 1$ sono 1 o indifferenze, e resta il termine $START \cdot L$ della cella di wait: $J_Z = X + Y + START \cdot L$. Scrivere $J_Z = 1 + START \cdot L$ vorrebbe dire $J_Z = 1$, e la macchina lascerebbe lo stato wait anche senza $START$.

Negli appunti lo stato stop ha solo un anello su se stesso: la lavatrice si ferma fino al reset. Va confermato se la specifica prevedeva il ritorno in wait (per esempio quando $START$ torna a 0).

Trasmettitore seriale

Specifica. Ingressi GO, D_0, D_1; uscita LN (la linea). Alla transizione di GO da 0 a 1 la macchina trasmette su LN il pacchetto $1, D_0, D_1, 0$, un bit per periodo: un bit di start a 1, i due bit di dato, un bit di stop a 0.

Se D_0 e D_1 non cambiano durante la trasmissione, la macchina li legge mentre li trasmette. Con Moore servono sette stati: init (attesa, rombo su GO), s (start: LN), due stati per il bit D_0 (uno con LN e uno senza, scelti da un rombo su D_0), due per D_1, e lo stato di stop, in cui si attende che GO torni a 0 prima di rientrare in init. Con Mealy bastano cinque stati, perché i bit di dato diventano uscite condizionate: in b0 l'ovale LN dopo il rombo su D_0, in b1 l'ovale LN dopo il rombo su D_1.

Se D_0 e D_1 possono cambiare dopo la transizione di GO, vanno letti tutti e due nell'istante in cui GO sale. Nell'ASM, dopo il rombo su GO ci sono i rombi su D_0 e su D_1, che separano quattro rami (00, 01, 10, 11), ciascuno con la sua sequenza fissa di stati con e senza LN. Il numero di stati cresce con il numero di bit; con pacchetti più lunghi conviene memorizzare i dati in un registro, ed è l'idea del paragrafo 4.9.

4.9 Controllore e datapath

Aggiunta in revisione · fonte: Donzellini e altri, *Introduzione al progetto di sistemi digitali*, e specifica dagli esercizi (RL-07)

Un sistema digitale appena più grande degli esempi precedenti si divide in due parti. Il **datapath** contiene i blocchi che elaborano e memorizzano i dati: registri, contatori, sommatori, MUX. Il **controllore** è una MSF che non tocca i dati ma comanda il datapath con i **segnali di controllo** (carica, azzera, abilita, scegli) e decide che cosa fare in base ai **segnali di stato** che il datapath gli restituisce (il terminal count di un contatore, l'esito di un confronto). Così la MSF resta piccola anche quando i dati sono molti bit: il numero di stati dipende dai passi dell'algoritmo, non dai valori dei dati.

Esempio: misura della durata di un impulso. Specifica: l'ingresso IN vale di solito 0 e ogni tanto fa un impulso lungo da 1 a 7 periodi di clock. Il sistema misura la durata di ogni impulso e la trasmette sulla linea seriale SER con il pacchetto: bit di start 1, Q_0, Q_1, Q_2, bit di stop 0. Il datapath ha un contatore CNT a 3 bit con abilitazione EN e azzeramento sincrono CL, e un MUX che porta su SER la costante 1, uno dei bit Q_i o la costante 0. Il controllo è diviso in due macchine (figura 4.7).

- **FSM-A** misura. Nello stato A0 attiva BSY (il sistema è alla ricerca di un impulso) e, finché $IN = 0$, l'uscita condizionata CL tiene il contatore a zero; quando $IN = 1$ attiva l'uscita condizionata EN, così il primo periodo dell'impulso è già contato, e passa in A1. In A1, finché $IN = 1$, attiva EN; quando IN torna a 0 passa in A2, che attiva TRG per un periodo: in $Q_2Q_1Q_0$ c'è la durata. In A3 aspetta ETX e poi torna in A0.
- **FSM-B** trasmette. Nello stato B0 aspetta TRG; poi cinque stati B1…B5 scelgono sul MUX, nell'ordine, 1, Q_0, Q_1, Q_2, 0. In B5 attiva anche ETX, per dire a FSM-A che il dato è stato inviato, e torna in B0.

Il contatore non cambia durante la trasmissione, perché FSM-A in A3 non attiva né EN né CL: è il motivo dello scambio TRG/ETX fra le due macchine.

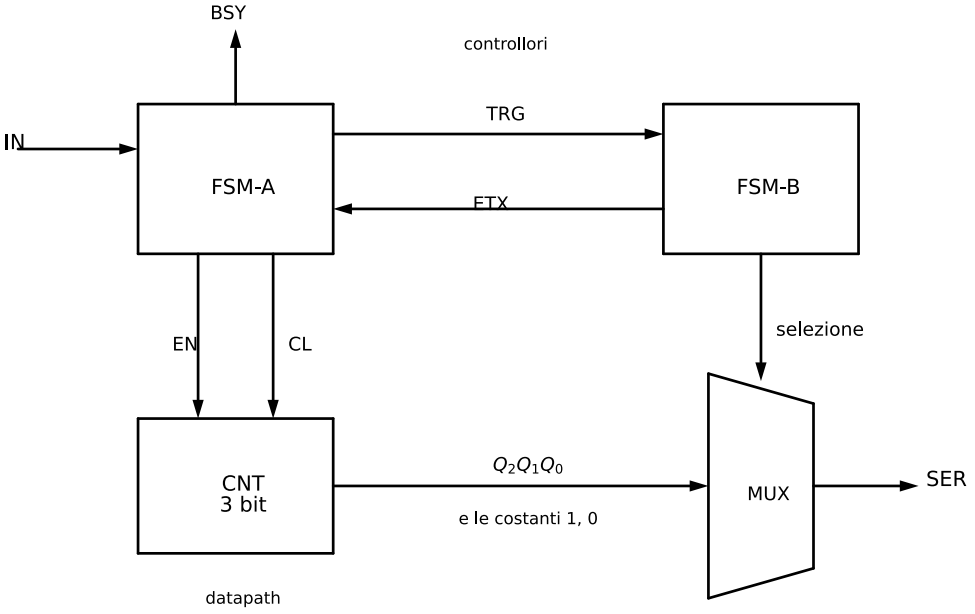


Figura 4.7

4.10 Simulazione

Aggiunta in revisione · fonte: obiettivi del corso e Donzellini e altri, *Introduzione al progetto di sistemi digitali* (RL-08)

Prima di costruire un circuito lo si verifica con un **simulatore**, un programma che calcola le uscite di uno schema o di un ASM per una sequenza di ingressi assegnata. Il procedimento è: (1) disegnare il circuito o l'ASM; (2) preparare gli stimoli, cioè il clock, il reset all'inizio e gli ingressi, facendoli cambiare lontano dai fronti; (3) scrivere a mano, prima di simulare, il diagramma temporale atteso; (4) confrontare il risultato della simulazione con quello atteso e spiegare ogni differenza. Conviene provare anche i casi limite: un impulso più breve di un periodo, due ingressi che cambiano insieme, il comportamento dopo il reset e dai codici non usati (che si possono forzare con i preset). Gli esercizi del corso con il simulatore chiedono proprio di completare a mano le sequenze temporali, come nell'esempio del registro con retroazione del paragrafo 3.7.

Esempi ed esercizi svolti

Esercizio 1. Verificare le equazioni di M1 (paragrafo 4.6). Dallo stato a (00): con $ENB = 1$ e $SEL = 1$, $D_X = 1 \cdot (0 + 1 \cdot 1) = 1$ e $D_Y = 1 \cdot (0 + 0) = 0$, quindi si va in c (10, GO); con $SEL = 0$, $D_X = 0$ e $D_Y = 1 \cdot (0 + 1 \cdot 1) = 1$, quindi si va in b (01, HLT). Da b con $ENB = 1$: $D_X = 1 \cdot (0 + SEL \cdot 0) = 0$, $D_Y = 1 \cdot (1 + \dots) = 1$, si resta in b qualunque sia SEL. Con $ENB = 0$ entrambe le equazioni danno 0 e si torna in a.

Esercizio 2. Che cosa fa la macchina delle luci se all'accensione si trova nello stato 110? Dalle equazioni dei FF D: $X^+ = X\bar{Z} + YZ = 1$, $Y^+ = Y\bar{Z} + \bar{X}\bar{Y}Z = 1$, $Z^+ = \bar{Z} = 1$, quindi va in 111; da 111, $X^+ = 1$, $Y^+ = 0$, $Z^+ = 0$, quindi va in 100 (stato e) e da lì prosegue normalmente. Per le uscite, in 110 si ha $A = B = 1$, $C = 1$, e in 111 lo stesso: per due periodi le luci sono tutte accese.

Esercizio 3. Progettare il rilevatore di fronti di Mealy. Un FF D memorizza il valore precedente dell'ingresso: $D = IN$, stato X. L'uscita è $OUT = IN \oplus X$: subito dopo un fronte di IN e fino al fronte successivo del clock, IN e X sono diversi e $OUT = 1$; al fronte del clock X si aggiorna e OUT torna a 0. L'impulso di uscita comincia con il fronte di IN e non con il clock, quindi dura meno di un periodo: è la differenza con la versione di Moore.

Riepilogo

Ogni rete sequenziale sincrona è una macchina a stati finiti, con un registro di stato, una rete dello stato successivo e una rete delle uscite; nella macchina di Moore le uscite dipendono solo dallo stato, in quella di Mealy anche dagli ingressi. Il diagramma ASM la descrive con blocchi di stato, blocchi decisionali e uscite condizionate, e ogni periodo di clock percorre un blocco ASM. Il progetto va dalla specifica all'ASM, dalla scelta e codifica degli stati alla tabella degli stati e alla sintesi degli ingressi dei FF, D o JK; l'analisi fa il percorso inverso. Sincronizzatore, rilevatore di fronti, rilevatore di impulso lungo e generatore di sequenze sono le strutture che ricorrono; Moore e Mealy risolvono gli stessi problemi con tempi e numeri di stati diversi, e una specifica ambigua può dare macchine non equivalenti. I codici non usati vanno controllati perché la macchina non vi resti intrappolata. Nei sistemi più grandi una MSF fa da controllore di un datapath fatto di registri e contatori, e ogni progetto si verifica con il simulatore confrontando il diagramma temporale ottenuto con quello atteso.